

Gymnázium J. K. Tyla

AI-asistovaný softwarový vývoj – Agentické inženýrství a harness DarkFactory

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Rozsah práce: 7127 slov / 47085 znaků
Rozsah práce: 7127 slov / 47085 znaků

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Odborná práce zkoumá využití agentní umělé inteligence při vývoji softwaru se zaměřením na architekturu agentního harnessu. Teoretická část vysvětluje jazykový model, inferenci a Harness; praktická část popisuje Agentické inženýrství a jeho realizaci v DarkFactory. Výsledky a diskuse vyhodnocují dostupné důkazy z implementace, testů, CI a cílových repozitářů. Evaluace používá zdrojový kód, automatické testy a CI výsledky a výslovně rozlišuje prokázané mechanismy od neprokázaného plného produkčního průchodu.

Abstract

This thesis examines the use of agentic artificial intelligence in software development, focusing on the architecture of an agent harness. The theoretical part explains the language model, inference, and the harness; the practical part describes Agentic Engineering and its realization in DarkFactory. Results and discussion evaluate the available evidence from implementation, tests, CI, and target repositories. The evaluation uses source code, automated tests, and CI results and explicitly distinguishes demonstrated mechanisms from a full production lifecycle that was not demonstrated.

Klíčová slova

.agents/, .claude/, Agentní sezení (Session), Agentní smyčka (Agent Loop), AGENTS.md, CLAUDE.md, Cílené smyčky (Goal Loops), Degradace kontextu (Context Rot), Dovednosti (Skills), Graf pracovního postupu (Workflow Graph), Guardrail, Harness, Hooks, Inferenční engine (Inference Engine), Integrovaný test (Integration Test), Izolované prostředí (Sandbox), Kompakce kontextu (Context Compaction), Kontextové inženýrství (Context Engineering), Kontextové okno (Context Window), MCP, Mezipaměť klíčů a hodnot (KV Cache), Nástroje (Tools), Plánování (Planning), Poskytovatel modelu (Model Provider), Prompt Injection, Promptové inženýrství (Prompt Engineering), Prostředí agenta (Agent Environment), Průběžná integrace (CI), Pull Request, Předání řízení (Handoff), Přepis (Transcript), RAG, Revize (Review), Slop, Spouštění kódu (Code Execution), Správa verzí (Version Control), Stav (State), Subagent, Swarm, Systémový prompt (System Prompt), Teplota (Temperature), Token, Transformer, Vektorová reprezentace (Embedding), Velký jazykový model (LLM), Vibe Coding, Vkládání kontextu (Context Injection), Vyvolávání nástrojů (Tool Calling), Vývoj řízený specifikací (Spec-Driven Development), Větev (Branch), Člověk ve smyčce (HITL)

Obsah

Prohlášení	2
Anotace	3
Abstract	4
Klíčová slova	5
1 Úvod	8
1.1 Motivace a vymezení problému	8
1.2 Východisko a argument práce	11
1.3 Cíle	11
1.3.1 Hlavní cíl	12
1.3.2 Dílčí cíle	12
1.4 Výzkumné otázky	12
1.5 Metodika	13
1.6 Struktura práce	13
2 Teoretická část	14
2.1 Jazykový model	14
2.1.1 Architektura a reprezentace	14
2.1.2 Inference	17
2.2 Harness	20
2.2.1 Smyčka a stav	21
2.2.2 Prostředí a nástroje	23
2.2.3 Rozšíření	26
3 Praktická část	30
3.1 Agentické inženýrství	30
3.1.1 Zadání a způsob práce	30
3.1.2 Řízení změny	31
3.1.3 Kvalita a ověřování	31
3.1.4 Instrukce a kontext	32
3.1.5 Řízení agentního chování	34
3.1.6 Orchestrace agentů	35
3.2 DarkFactory	36
4 Výsledky a diskuse	37
4.1 Ověření mechanismů	37
4.2 Ověření systému	37
4.3 Ověření na repozitářích	37

4.4 Výzkumné otázky	38
4.5 Diskuse a omezení	38
5 Závěr	40
Seznam zdrojů	41
Seznam obrázků a tabulek	48
Seznam příloh	50
A Encyklopedie a rejstřík pojmů	51

1 Úvod

[

Práce se zabývá použitím agentní AI při vývoji softwaru a technickými podmínkami, které umožňují delegovat delší úlohy bez ztráty kontroly nad stavem, účinky a ověřením výsledku.]

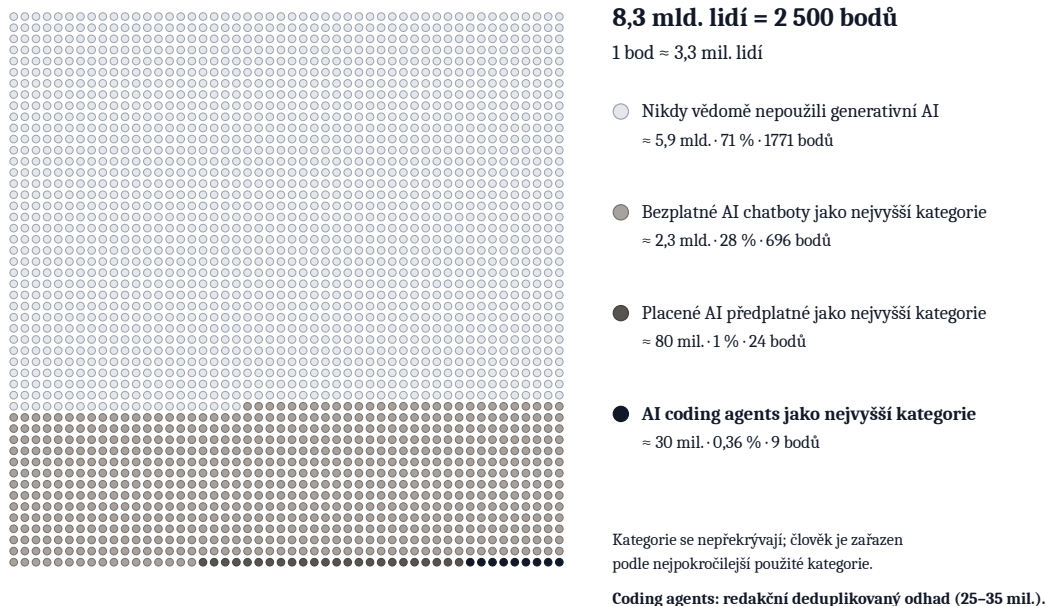
Předmětem práce není trénování jazykových modelů, ale systémové vrstvy potřebné pro jejich praktické použití jako součásti agentního vývojového procesu. Východisko a argument, podle kterého je tato otázka dále rozpracována, stanovuje část 1.2.

1.1 Motivace a vymezení problému

[

Rozsah využití generativní AI a rychlý vývoj modelových schopností vytvářejí nové možnosti pro vývoj softwaru. Samotná dostupnost schopného modelu však neurčuje, jak je softwarová práce zadána, provedena, ověřena a řízena.]

Používání generativní AI je už globálně rozšířené. Gradually ve svém srpnovém odhadu z roku 2026 rozděluje světovou populaci do čtyř vzájemně výlučných kategorií podle nejpokročilejšího způsobu používání AI: 1 771 z 2 500 bodů připadá na lidi, kteří generativní AI nikdy vědomě nepoužili, 696 na uživatele bezplatných chatbotů, 24 na placící uživatele a 9 na pravidelné uživatele AI coding agents. ⁽¹⁾



Obrázek 1: Odhad rozdělení světové populace podle nejpokročilejší používané kategorie generativní AI v srpnu 2026. Každý z 2 500 bodů představuje přibližně 3,3 milionu lidí; kategorie jsou vzájemně výlučné. Hodnota pro pravidelné uživatele AI coding agents je redakční, deduplikovaný odhad Gradually (25–35 milionů), přičemž graf používá střed 30 milionů, přibližně 0,36 % populace. ⁽¹⁾

Poslední kategorie nesmí být čtena jako globální sčítání uživatelů. Gradually ji výslovně uvádí jako redakční, deduplikovaný odhad 25–35 milionů pravidelných uživatelů; vizualizace používá střed 30 milionů, tedy přibližně 0,36 % světové populace. ⁽¹⁾ Rozdíl mezi širokým používáním generativní AI a podstatně menším odhadovaným počtem uživatelů coding agents ukazuje, že pokročilé agentní použití zůstává relativně úzkou podmnožinou celkové adopce.

Jedním z rozpoznatelných způsobů AI-asistovaného programování je Vibe Coding: způsob tvorby softwaru, při kterém člověk iteruje pomocí pokynů v přirozeném jazyce bez průběžné kontroly vygenerovaného kódu. ⁽²⁾

Termín zavedl Andrej Karpathy v roce 2025. Jeho popis představuje nízkostrukturovaný způsob interakce s generativním systémem a poskytuje užitečný kontrast k postupu, který před delegováním práce explicitně formuluje specifikaci, plán a podmínky ověření. ^(2, 3) Existence tohoto způsobu práce sama o sobě nevypovídá o tom, jak velká část uživatelů coding agents takto postupuje.

Tweet Andreje Karpathyho z 2. února 2025, ve kterém popsal původní význam Vibe Coding.



Andrej Karpathy ✓
@karpathy



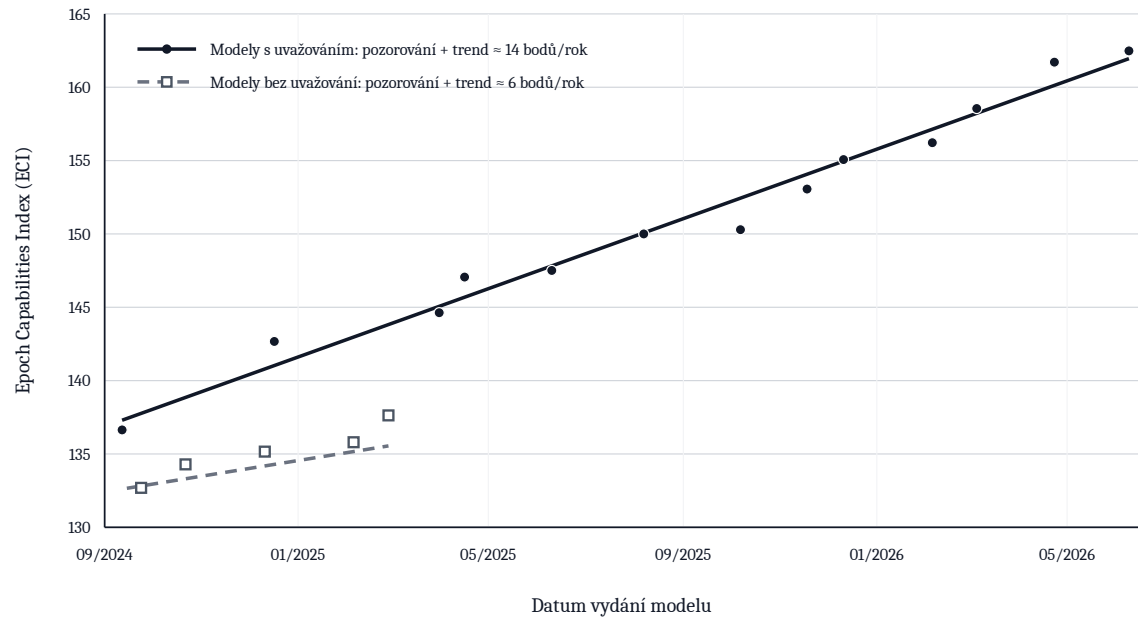
There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

3:17 PM · Feb 2, 2025 · **6.8M** Views

Obrázek 2: Původní tweet Andreje Karpathyho o Vibe Coding. ^(2, 4)

Takový způsob práce může zrychlit průzkumné prototypování, ale bez explicitních kontrol zvyšuje význam následné revize, testů a sledovatelnosti změn. V této práci proto Vibe Coding slouží jako současný kontrast k záměrně strukturovanému agentickému inženýrství, nikoli jako popis celé populace uživatelů AI pro programování.

Současně se zvyšují schopnosti samotných modelů. Epoch AI na datech Epoch Capabilities Index (ECI) zpřístupněných k 1. září 2026 odhaduje po nástupu reasoning modelů v září 2024 tempo posunu frontier přibližně 14 ECI bodů za rok, zatímco pro non-reasoning frontier přibližně 6 bodů za rok. ECI je kompozitní index skládající více benchmarků do jedné škály schopností; nejde o univerzální měřítko inteligence. ⁽⁵⁾



Obrázek 3: Frontier pozorování a lineární trendy Epoch Capabilities Index (ECI) od přechodu k reasoning modelům v září 2024. Epoch uvádí přibližně 14 ECI bodů za rok pro reasoning frontier a přibližně 6 bodů za rok pro non-reasoning frontier; zdrojová data jsou ve verzi z 1. 9. 2026. Pro čitelnost v tisku nejsou zobrazeny 90% predikční intervaly. ⁽⁵⁾

Artificial Analysis Intelligence Index v4.3.2 poskytuje bodový snímek současných modelových schopností napříč deseti evaluacemi, mezi nimi Terminal-Bench 4.0 a SciCode. Jde o kompozitní benchmark, nikoli o univerzální pořadí modelů pro každé použití. ⁽⁶⁾

Model	Poskytovatel	Artificial Analysis Intelligence Index	Terminal-Bench 4.0	SciCode
Claude Fable 5.1 (Max, default fallback)	Anthropic	53	52 %	63 %
GPT-6 Astra (max)	OpenAI	53	59 %	56 %
Claude Opus 5 (Max)	Anthropic	51	49 %	56 %
Claude Fable 5 (Max, Opus 4.8 fallback)	Anthropic	50	42 %	61 %
Muse Spark 1.3 (max)	Meta	48	33 %	59 %

Tabulka 1: Bodový snímek pěti nejvýše skórujících odlišných základních modelů podle Artificial Analysis Intelligence Index v4.3.2, zveřejněného 7. 9. 2026; hodnoty byly ověřeny 22. 9. 2026. U každého základního modelu je ponechána jeho nejvýše skórující vyhodnocená konfigurace. ⁽⁶⁾

Dílčí výsledky ukazují rozdílné profily schopností. Claude Fable 5.1 (Max, default fallback) a GPT-6 Astra (max) mají v tomto snímku shodný agregovaný index 53, ale GPT-6 Astra dosahuje vyššího výsledku v Terminal-Bench 4.0 (59 % oproti 52 %), zatímco Claude Fable 5.1 dosahuje vyššího výsledku v SciCode (63 % oproti 56 %). ⁽⁶⁾ Agregované pořadí a pořadí na jednotlivých benchmarkech se tedy mohou lišit, protože modelová schopnost je vícerozměrná.

Epoch i Artificial Analysis zachycují modelové schopnosti prostřednictvím benchmarků a trendů. Takové výsledky samy o sobě neprokazují spolehlivost reálného softwarově-inženýrského workflow: benchmark modelu nevymezuje perzistentní stav úlohy, provedení nástrojů, interakci s prostředím, ověření účinků, správu kontextu, orchestraci ani způsob řízení změny. ^(7,8)

Současné agentní systémy proto stavějí kolem modelové inference další běhovou a řídicí vrstvu, která propojuje model se stavem, nástroji, prostředím a kontrolními mechanismy. ⁽⁹⁾ ¹⁰⁾ Technický problém práce tak neleží pouze ve schopnosti modelu generovat kód, ale v tom, jak tyto okolní mechanismy záměrně navrhnout a použít pro delší, ověřitelnou softwarovou práci.

Práce na tento problém odpovídá ve čtyřech navazujících krocích: teoretická část vysvětluje model, inferenci a Harness; praktická část vymezuje postupy Agentického inženýrství; DarkFactory tyto postupy konkrétně realizuje; a část Výsledky a diskuse hodnotí dostupné implementační a provozní důkazy.

1.2 Východisko a argument práce

[
Východiskem práce je návaznost model → Harness → Agentické inženýrství → DarkFactory.]

Jazykový model poskytuje inferenci nad aktivním tokenovým kontextem. ⁽¹¹⁾ Harness z jednotlivých inferenčních kroků vytváří agentní runtime tím, že udržuje stav, opakuje smyčku, zprostředkovává nástroje a prostředí a poskytuje mechanismy rozšíření. ⁽¹⁰⁾

Agentické inženýrství určuje, jak jsou tyto schopnosti záměrně použity pro softwarovou práci: jak se formuluje zadání, řídí změna, spravuje kontext, ověřuje výsledek a koordinuje více kroků nebo agentů. DarkFactory je implementační artefakt, na kterém jsou tyto postupy realizovány a následně vyhodnoceny. Úplné definice jednotlivých mechanismů jsou ponechány příslušným teoretickým a praktickým částem.

1.3 Cíle

[
Tato část stanovuje hlavní a dílčí cíle.]

Cíle vymezují, jaký artefakt má být navržen a které jeho vlastnosti mají být technicky ověřeny. Výzkumné otázky jsou odděleny do následující samostatné části.

1.3.1 Hlavní cíl

[

Navrhnout a implementovat architekturu Harnessu pro dlouhotrvající vývoj softwaru, která odděluje jazykový model od trvalého stavu, prostředí a deterministických kontrolních mechanismů, a technicky ověřit vlastnosti této architektury na systému DarkFactory.]

DarkFactory je implementační artefakt této architektury. Splnění cíle se posuzuje podle dohledatelné implementace navržených mechanismů a reprodukovatelných testovacích nebo provozních důkazů. Úplný živý životní cyklus změny je samostatná úroveň ověření a nesmí být zaměněn za samotnou existenci architektury nebo úspěšné komponentové testy.

1.3.2 Dílčí cíle

[

- Vymezit teoretické mechanismy AI-asistovaného vývoje, jazykového modelu a inference, Harnessu a Agentického inženýrství potřebné pro dlouhotrvající agentní vývoj softwaru.
- Navrhnout a implementovat DarkFactory s explicitním stavem běhu, odděleným prostředím, rozšiřitelným capability rozhraním, GitHubem jako řídicí vrstvou a oddělenými hranicemi lidské a strojové identity.
- Implementovat mechanismy řízeného životního cyklu změny: plánování, deterministické ověření, smyčku revize a opravy, finální kontrolu souladu, integraci a obnovu přerušného běhu.
- Ověřit implementované mechanismy automatickými testy a CI nad konkrétním commitem.
- Ověřit přenositelnost vybraných částí řešení na konkrétních cílových repozitářích a samostatně vyhodnotit, zda existuje důkaz úplného živého životního cyklu změny.
- Vztáhnout zjištěné výsledky a jejich omezení přímo k výzkumným otázkám.

]

Dílčí cíle oddělují teoretické vymezení, konstrukci artefaktu a jednotlivé úrovně jeho ověření. Nesplněná nebo neprokázaná úroveň evaluace proto nemusí být nahrazena silnějším tvrzením z jiné vrstvy důkazů.

1.4 Výzkumné otázky

[

- O1: Které mechanismy agentního harnessu a vývojového životního cyklu umožňují agentovi samostatně provádět softwarovou změnu, zatímco rozhodnutí s vyšším dopadem zůstávají explicitně řízena člověkem?
- O2: Jak může agentní harness omezit nebo obnovit neproduktivní či přerušný běh bez ztráty již ověřeného stavu a bez opakování přijatých deterministických účinků?
- O3: Jak lze oddělit trvalý stav dlouhotrvající úlohy od omezeného kontextového okna modelu tak, aby bylo možné práci po přerušení bezpečně obnovit a pokračovat v ní?

]

O1 sleduje řízenou autonomii, O2 odolnost provádění a O3 kontinuitu stavu přes hranice jednotlivých modelových kontextů a běhů. Otázky jsou záměrně formulovány tak, aby na ně bylo možné odpovědět konkrétními architektonickými prvky a reprodukovatelnými důkazy z implementace.

1.5 Metodika

[
Práce kombinuje rešerši odborné literatury, technických specifikací a primární dokumentace s konstrukčním přístupem odpovídajícím design science. ^(12, 13)]

Nejprve jsou v teoretické části vysvětleny mechanismy jazykového modelu, inference a Harnessu v rozsahu potřebném pro porozumění agentnímu systému. Na ně navazuje syntéza postupů Agentického inženýrství pro zadávání, řízení změny, ověřování, práci s kontextem a orchestraci.

Tyto postupy jsou následně realizovány v systému DarkFactory. Vyhodnocení používá jako důkaz zdrojový kód, automatické testy, CI a dohledatelné artefakty z cílových repozitářů; síla každého závěru je omezena na rozsah skutečně dostupných reprodukovatelných důkazů.

1.6 Struktura práce

Kapitola 2, Teoretická část, vysvětluje fungování jazykového modelu, inference a Harnessu. Kapitola 3, Praktická část, popisuje postupy Agentického inženýrství a jejich konkrétní realizaci v DarkFactory. Kapitola 4 shrnuje výsledky, odpovídá na výzkumné otázky a diskutuje omezení dostupných důkazů. Kapitola 5 práci uzavírá ve vztahu k jejím cílům a zjištěním.

2 Teoretická část

Teoretická část vymezuje mechanismy potřebné k porozumění tomu, jak agentní systém pracuje. Jazykový model a inference vysvětlují vznik modelového výstupu; Harness vysvětluje běhovou vrstvu, která zajišťuje kontinuitu, nástroje a interakci s prostředím.

2.1 Jazykový model

[
Jazykový model převádí tokenový kontext na inferenční výstup; výsledné chování proto závisí jak na reprezentaci vstupu a architektuře modelu, tak na způsobu provedení inference a jejích runtime omezeních. ^(11, 14)]

Modelová vrstva končí vytvořením výstupu nad aktuálním kontextem. Trvalý stav úlohy, účinky v externím prostředí, nástroje a dlouhodobé řízení patří až do Harnessu.

2.1.1 Architektura a reprezentace

Velký jazykový model (LLM)

[
Velký jazykový model (LLM) je parametrický model pravděpodobnostního rozdělení nad posloupnostmi tokenů; autoregresivní LLM generuje pokračování postupným odhadem dalšího tokenu z již dostupného kontextu. ⁽¹¹⁾]

Při inferenci model převádí aktivní posloupnost tokenů na distribuci možných pokračování a z ní vytváří výstup. Současné generativní LLM jsou často založeny na architektuře **Transformer**, ale modelová inference sama nepředstavuje perzistentní pracovní stav, vykonání nástroje ani změnu externího prostředí. ⁽¹⁴⁾

GPT-3. Brown et al. popisují GPT-3 jako autoregresivní jazykový model se 175 miliardami parametrů, který při evaluaci provádí zero-shot, one-shot a few-shot úlohy pouze z textového kontextu bez gradientních aktualizací vah. ⁽¹¹⁾

[*Praktický význam:* Model dodává inferenční schopnost potřebnou pro generování, klasifikaci nebo volbu dalšího kroku. Perzistentní stav workflow, skutečné vykonání nástrojů, účinky v prostředí a dlouhodobá orchestrace proto musí vzniknout mimo samotný model.]

Transformer

[
Transformer je architektura neuronové sítě, která zpracovává vztahy v sekvenci pomocí mechanismů pozornosti místo rekurence či konvoluce jako základního mechanismu pro přenos informace mezi pozicemi. ⁽¹⁴⁾]

Původní architektura Transformer má enkodér a dekodér; autoregresivní chování vzniká v dekodéru maskováním přístupu k budoucím tokenům a postupným vytvářením dalšího výstupu. Transformer tedy není synonymem pro autoregresivní LLM, ale jeho pozornostní mechanismus tvoří základ zpracování aktivního kontextu v mnoha současných jazykových modelech. ⁽¹⁴⁾

Původní Transformer. Vaswani et al. demonstrují Transformer na strojovém překladu: enkodér i dekodér skládají vrstvy pozornosti a dopředných sítí, přičemž maskovaná self-attention v dekodéru brání přístupu k budoucím pozicím během autoregresivní predikce. ⁽¹⁴⁾

[*Praktický význam:* Pozornost umožňuje, aby výpočet dalšího výstupu podmiňovaly informace z aktivního kontextu. Informace, které mají přetrvat mezi inferenčními běhy nebo mimo dostupný kontext, však musí udržovat okolní systém.]

Tokenizér

[
Tokenizér je komponenta, která podle slovníku a segmentačních pravidel mapuje vstup na posloupnost diskrétních tokenů a jejich identifikátorů a umožňuje odpovídající zpětné dekodování. ⁽¹⁵⁾]

Subword tokenizace dělí text na jednotky menší než celé slovo, takže model nepotřebuje samostatnou položku slovníku pro každé možné slovo. BPE postupně slučuje časté sousední jednotky a vytváří omezený slovník, z něhož lze skládat i dříve neviděná slova. ⁽¹⁵⁾

Subword BPE. Sennrich et al. používají Byte Pair Encoding pro překlad angličtiny do němčiny a ruštiny tak, aby omezený slovník reprezentoval otevřenou slovní zásobu sekvencemi subword jednotek; v analýze uvádějí německý kompozit „Sonnensystem“ složený z „Sonne“ a „System“. ⁽¹⁵⁾

[*Praktický význam:* Zvolená tokenizace určuje, kolik tokenů spotřebují instrukce, historie, zdrojový kód i výsledky nástrojů. Stejný text tak může podle tokenizéru zabírat odlišnou část vstupní nebo výstupní kapacity modelu.]

Token

[
Token je diskrétní jednotka sekvence identifikovaná položkou slovníku tokenizéru; podle tokenizační metody může odpovídat celému slovu, části slova nebo jinému textovému fragmentu. ⁽¹⁵⁾]

Token se od tokenizéru liší tím, že je výslednou sekvencí jednotkou, zatímco tokenizér určuje pravidla jejího vzniku. Před vstupem do vrstev Transformeru se identifikátory vstupních tokenů mapují na spojitě naučené vektory. ⁽¹⁴⁾

Token pod úrovní slova. Subword přístup Sennricha et al. reprezentuje vzácná a neznámá slova jako posloupnost menších jednotek namísto jediné položky celého slova; rozbor

německých kompozit, například „Sonnensystem“ („Sonne“ + „System“), ukazuje, proč token nemusí odpovídat jednomu slovu. ⁽¹⁵⁾

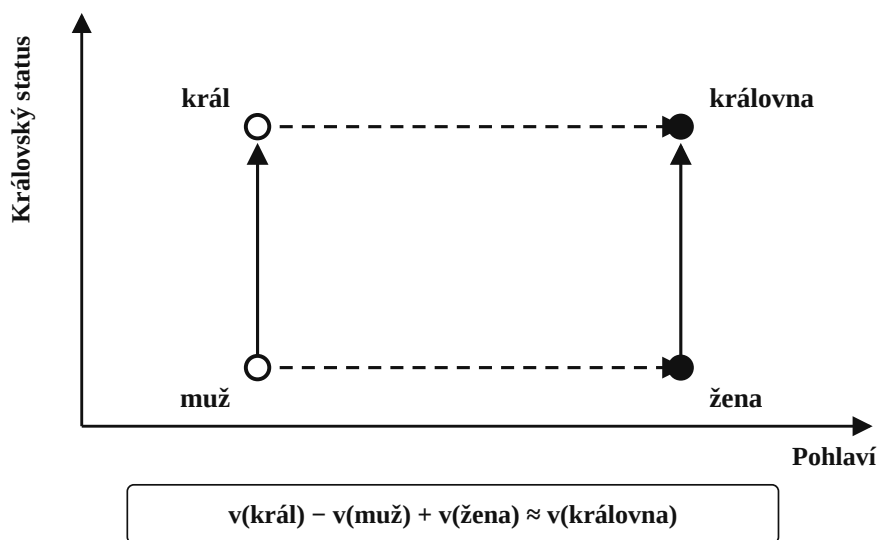
[*Praktický význam:* Tokeny jsou jednotkou, podle níž se vyjadřuje délka aktivního kontextu a generovaného výstupu. Agentní systém proto musí sledovat, kolik tokenů zabírají instrukce, historie, data i nástrojové výsledky.]

Vektorová reprezentace (Embedding)

[Embedding je spojitá vícerozměrná vektorová reprezentace diskrétního prvku, v níž se naučené geometrické vztahy mohou využít k zachycení podobnosti a dalších vztahů mezi reprezentovanými objekty. ⁽¹⁶⁾]

Uvnitř Transformeru se identifikátory tokenů mapují na naučené vektory, které vstupují do neuronového zpracování. ⁽¹⁴⁾ Vektorové reprezentace však lze používat i mimo samotnou inferenci jazykového modelu, například pro porovnání blízkosti reprezentací; jde o obecnější použití než interní tokenové embeddingy. ⁽¹⁶⁾

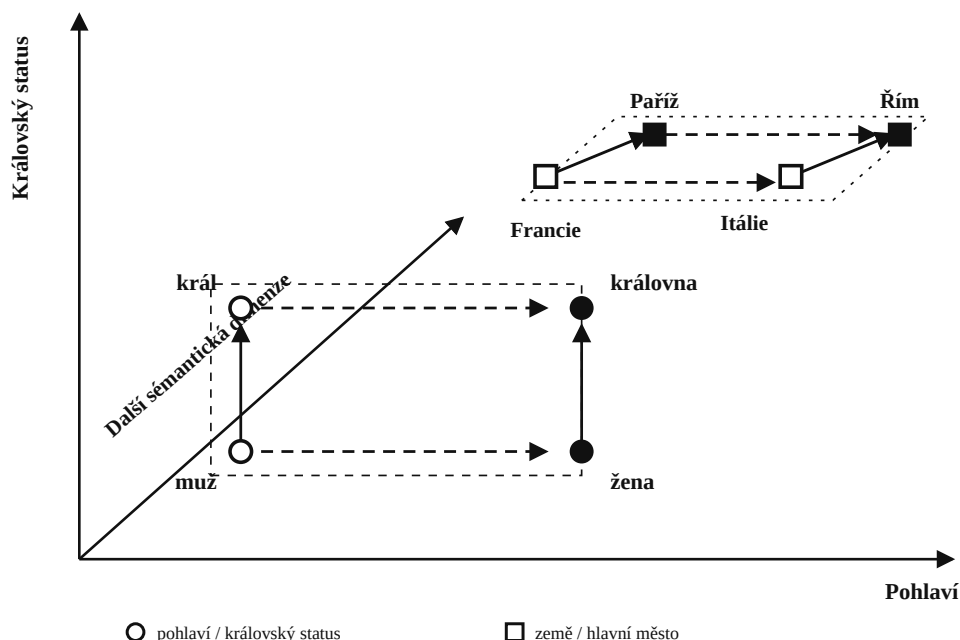
Analogie král – muž + žena ≈ královna. Mikolov, Yih a Zweig ukazují, že rozdílové vektory naučených slovních reprezentací mohou zachycovat sémantické vztahy; klasickým příkladem je vektor blízký vztahu „King – Man + Woman ≈ Queen“. ⁽¹⁷⁾



Obrázek 4: Ilustrativní 2D projekce vztahu král – muž + žena ≈ královna. Osy Pohlaví a Královský status jsou vysvětlující projekcí pro názornost, nikoli doslovnými naučenými dimenzemi nebo produkčními souřadnicemi embeddingového prostoru.

Dvě relační rodiny v projekci. Vedle vztahu pohlaví a královského statusu ukazují práce o word embeddings také relační rodinu země–hlavní město; Mikolov et al. uvádějí například vztah „Madrid – Spain + France ≈ Paris“. Druhá rovina proto pedagogicky zobrazuje analo-

gickou dvojici Francie–Paříž a Itálie–Řím bez tvrzení, že zakreslené souřadnice odpovídají skutečným naučeným osám. ^(17,18)



Obrázek 5: Pedagogická 3D projekce dvou relačních rodin. Reálné embeddingové prostory jsou vysokodimenzionální; osy Pohlaví, Královský status a Další sémantická dimenze jsou pouze vysvětlující projekční pomůcky a nepředstavují doslovné produkční souřadnice embeddingu.

[*Praktický význam:* Vektorové reprezentace umožňují řadit nebo vyhledávat položky podle sémantické podobnosti, což je užitečné při výběru relevantních informací pro další modelový krok. Samotný embedding přitom neurčuje, jak se vybraný kontext následně spravuje nebo používá.]

2.1.2 Inference

Poskytovatel modelu (Model Provider)

[Poskytovatel modelu je externí služba nebo programové rozhraní, přes které runtime vybírá a volá konkrétní model; provider může mapovat abstraktní jméno modelu na vlastní implementaci modelového API. ⁽¹⁹⁾]

Provider není samotný jazykový model ani lokální inferenční engine. Zprostředkovává přístup k modelům a určuje kontrakt požadavku, dostupné schopnosti a provozní omezení daného rozhraní; různí provideri proto mohou stejnou roli realizovat odlišnými API. ⁽¹⁹⁾

OpenAI Agents SDK. Dokumentace OpenAI Agents SDK ukazuje konfiguraci modelu řetězcem, například `RunConfig(model="gpt-5.6-sol")`; výchozí OpenAI provider tento název překládá na konkrétní modelové rozhraní založené na Responses API. ⁽¹⁹⁾

[*Praktický význam:* Volba poskytovatele určuje, ke kterým modelům a schopnostem má agentní runtime přístup, jaké požadavky musí vytvářet a s jakými limity musí počítat. Oddělená provider vrstva umožňuje měnit způsob přístupu bez změny významu samotného modelu.]

Inferenční engine (Inference Engine)

[
Inferenční engine je běhová vrstva, která načítá model a skutečně provádí jeho dopředné výpočty a autoregresivní generování nad vstupními tokeny. ⁽²⁰⁾]

Serving engine může kromě samotného výpočtu plánovat a dávkovat požadavky, spravovat akceleratorovou paměť a organizovat KV cache. Tím se liší od poskytovatele modelu: provider je přístupové rozhraní nebo služba, zatímco inference engine řeší fyzické provedení modelu a správu runtime prostředků. ⁽²¹⁾

vLLM a PagedAttention. vLLM používá mechanismus PagedAttention pro správu KV cache po blocích místo požadavku na jeden souvislý paměťový prostor; práce Kwon et al. popisuje tuto paměťovou správu jako součást serving systému pro LLM. ^(20, 21)

[*Praktický význam:* Vlastnosti inference engine ovlivňují latenci, propustnost, využití paměti a tím i cenu nebo proveditelnost opakovaných a dlouhotrvajících modelových volání. Tyto vlastnosti jsou důležité i tehdy, když je engine skryt za vzdáleným provider API.]

Teplota (Temperature)

[
Teplota je parametr vzorkování, který u rozhraní, jež jej podporují, mění koncentraci pravděpodobnostního výběru dalších tokenů a tím ovlivňuje variabilitu generovaného výstupu. ⁽²²⁾]

Nižší teplota typicky soustřeďuje výběr na pravděpodobnější pokračování, zatímco vyšší hodnota připouští větší variabilitu. Nelze ji ztotožnit s jednoduchým přepínačem determinismu: přesný rozsah, význam a interakce s dalšími sampling parametry jsou vlastností konkrétního modelového rozhraní. ⁽²²⁾

OpenAI Responses API. OpenAI u rozhraní Responses dokumentuje `temperature` v rozsahu 0 až 2 a jako příklady uvádí vyšší hodnotu 0,8 pro náhodnější výstup a nižší hodnotu 0,2 pro soustředěnější výstup. Jde o kontrakt tohoto konkrétního API, nikoli o univerzální rozsah všech modelů a providerů. ⁽²²⁾

[*Praktický význam:* Podporované sampling parametry lze volit podle charakteru úlohy: stabilnější strukturované kroky mohou vyžadovat koncentrovanější výběr, zatímco exploranční generování může využít větší variabilitu. Nastavení musí respektovat možnosti konkrétního API.]

Kontextové okno (Context Window)

[

Kontextové okno je konečný rozsah tokenové sekvence, kterou model může mít v daném inferenčním běhu současně k dispozici jako aktivní vstup. ⁽²³⁾]

O tuto kapacitu se dělí instrukce, historie konverzace, uživatelská data, výsledky nástrojů a další vložený obsah. Nominální maximální délka však popisuje kapacitu vstupu, nikoli záruku, že model využije každou relevantní informaci v dlouhém kontextu stejně spolehlivě. ⁽²³⁾

Long-context model s nominálním 16K oknem. Liu et al. mezi testovanými systémy zahrnují LongChat-13B s kontextovým oknem 16K a ukazují, že samotná deklarovaná kapacita neznamená stejně spolehlivé využití informace ve všech pozicích dlouhého vstupu. ⁽²³⁾

[*Praktický význam:* Agent nemůže do jednoho inferenčního kroku bez omezení hromadit instrukce, přepis, nástrojové výstupy a další kontext. Musí proto hlídat aktivní tokenový rozpočet a rozhodovat, které informace mají být v konkrétním kroku dostupné.]

Mezipaměť klíčů a hodnot (KV Cache)

[KV cache je runtime mezipaměť dříve vypočtených klíčů a hodnot pozornostních vrstev pro tokeny již zpracovaného prefixu, které lze znovu použít při autoregresivním dekodování dalších tokenů. ⁽²⁴⁾]

Opakované použití uložených klíčů a hodnot omezuje potřebu znovu počítat pozornostní reprezentace celého prefixu, ale cache současně spotřebovává paměť a její velikost roste s počtem aktivních tokenů a sekvencí. ⁽²¹⁾ KV cache je interní runtime mechanismus inference a není totéž co providerové prompt caching, billing cache ani sémantická cache.

Bloková správa KV cache ve vLLM. PagedAttention ve vLLM ukládá KV cache do nesouvislých bloků a mapuje logické bloky sekvence na fyzické bloky paměti. Tím řeší proměnlivou velikost cache jednotlivých sekvencí a umožňuje bezpečné sdílení bloků tam, kde je obsah společný. ⁽²¹⁾

[*Praktický význam:* KV cache zrychluje pokračující autoregresivní generování z již zpracovaného prefixu za cenu runtime paměti. Při dlouhých nebo souběžných agentních bězích proto může být správa cache významnou součástí kapacitního plánování inference.]

Degradace kontextu (Context Rot)

[Degradace kontextu (Context Rot) označuje praktický pokles spolehlivosti, s níž model dokáže využívat relevantní informace při růstu délky, informačního zatížení nebo nevýhodném umístění informace v aktivním kontextu. ⁽²³⁾]

Empirické studie dlouhého kontextu ukazují, že schopnost nalézt a využít relevantní údaj není určena pouze tím, zda se údaj vejde do nominálního kontextového okna. Výkon může záviset na jeho poloze a přidání dalšího kontextu proto samo o sobě nezaručuje spolehlivější využití všech vložených informací. ⁽²³⁾

„Lost in the Middle“. V experimentu s multi-document question answering i key-value retrieval vykazují testované dlouhokontextové modely výraznou citlivost na polohu relevantní informace: výkon bývá vyšší, když je informace na začátku nebo na konci vstupu, a nižší při jejím umístění uprostřed dlouhého kontextu. ⁽²³⁾

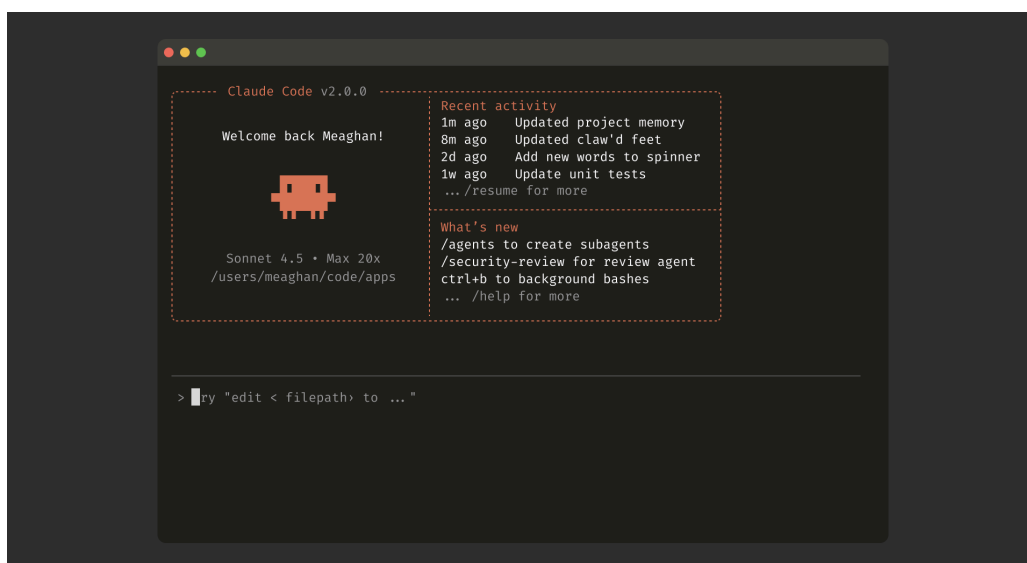
[Praktický význam: Pouhé hromadění celé historie není spolehlivou strategií pro dlouhotrvající agentní běhy. Pozdější vrstvy agentního systému proto potřebují kontext selektivně vybírat, zkracovat nebo jinak spravovat podle aktuálního kroku.]

Modelová inference poskytuje výstup z konečného aktivního kontextu, nikoli kontinuitu dlouhotrvající úlohy ani provedení účinků v externím prostředí. Systém, který má uchovávat stav, používat nástroje a pokračovat napříč více kroky, proto potřebuje další vrstvu: Harness.

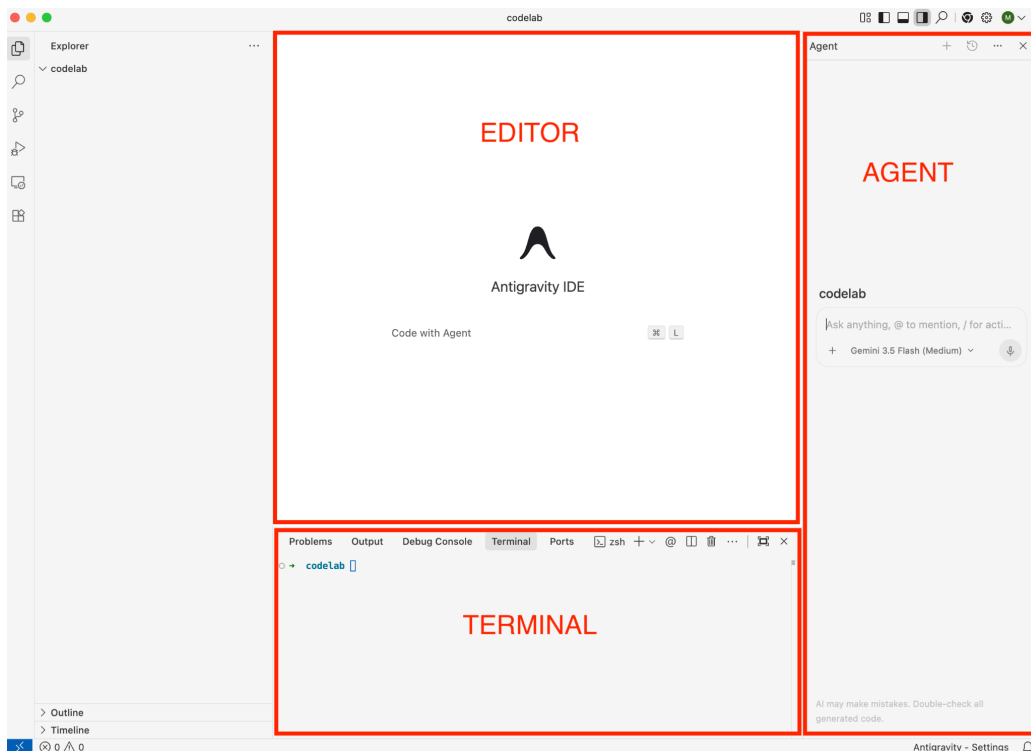
2.2 Harness

[Model provádí inferenci nad aktuálním vstupem; Harness je běhová vrstva kolem této inference, která z jednotlivých inferenčních kroků vytváří pokračující agentní běh. Opakuje kroky, zachovává kontinuitu sezení a stavu, zpřístupňuje vnější prostředí a nástroje, provádí vyžádané účinky mimo model, omezuje jejich prostředí a načítá rozšíření. OpenAI Agents SDK například poskytuje vestavěnou agentní smyčku, která zpracovává vyvolání nástrojů a vrací jejich výsledky modelu, a SandboxAgent s pracovním prostorem, shellem, souborovým systémem a obnovitelným stavem sandboxu. ^(25, 26)]

Harness zde vysvětluje dostupné běhové mechanismy, nikoli metodiku jejich záměrného skládání. Způsob specifikace úlohy, výběr kontextu, revize práce, orchestrace agentů a návrh CI či workflow patří do části 3.1 Agentické inženýrství.



Obrázek 6: Terminálové rozhraní Claude Code jako příklad uživatelské vrstvy agentního Harnessu: model je zpřístupněn uvnitř pokračujícího běhu s viditelným stavem práce a nástrojovými interakcemi. Snímek pochází z oficiálního materiálu Anthropic. ⁽²⁷⁾



Obrázek 7: Google Antigravity IDE jako příklad integrovaného agentního prostředí: editor, terminál a agentní panel tvoří rozhraní k pracovnímu prostoru, ve kterém může agent pozorovat a měnit software. Snímek pochází z oficiálního Google Codelabu. ⁽²⁸⁾

[*Praktický význam:* Harness převádí jednotlivé modelové inference na stavový agentní runtime, ve kterém mohou rozhodnutí modelu navazovat na skutečné výsledky předchozích akcí a působit v kontrolovaném prostředí.]

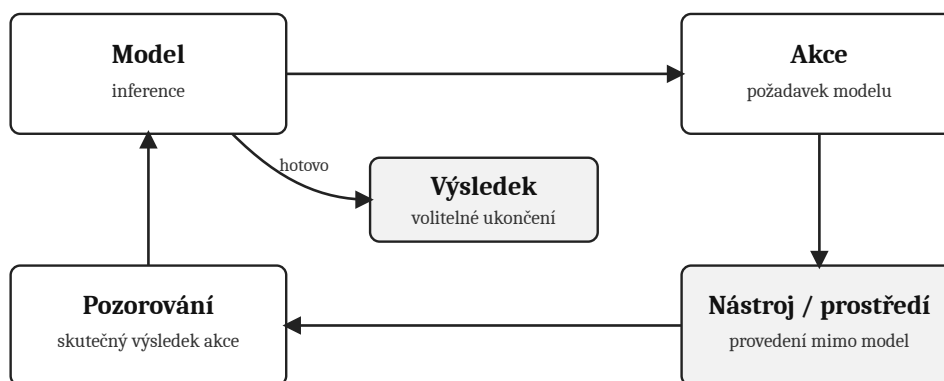
2.2.1 Smyčka a stav

Agentní smyčka (Agent Loop)

[Agentní smyčka je opakovaný běhový cyklus, ve kterém model z aktuálního vstupu vybere další akci nebo ukončení; vybranou externí akci provede Harness prostřednictvím nástroje či prostředí a vzniklé pozorování předá do další inference. ⁽²⁹⁾]

Základní mechanismus lze vyjádřit jako **Model** → **Akce** → **Nástroj/prostředí** → **Pozorování** → **Model**, s volitelnou koncovou větví **Model** → **Výsledek**. Model tedy rozhoduje o požadované akci, ale externí účinek nevykonává sám. Původní práce ReAct kombinuje uvažování a jednání tak, že akce interagují s vnějšími zdroji a jejich výsledky se stávají pozorováními pro další krok. ⁽²⁹⁾

ReAct nad Wikipedií. V úlohách HotpotQA a FEVER autoři ReAct používají jednoduché rozhraní k Wikipedii: model volí akce pro vyhledání či dohledání externí informace a získané pozorování následně ovlivňuje další inferenční krok. Jde o konkrétní instanci principu akce–pozorování, nikoli o univerzální architekturu všech agentních systémů. ⁽²⁹⁾



Obrázek 8: Schéma agentní smyčky inspirované interakcí ReAct: model volí akci, Harness ji předá nástroji nebo prostředí, pozorování výsledku se vrací modelu a cyklus může skončit výsledkem. ReAct je zde zdrojem mechanismu, nikoli tvrzením, že každý agentní systém používá totožnou architekturu. ⁽²⁹⁾

[*Praktický význam:* Smyčka umožňuje, aby pozdější rozhodnutí vycházela ze skutečně pozorovaných výsledků předchozích akcí místo z jednorázového odhadu modelu.]

Agentní sezení (Session)

[Agentní sezení je identifikační a kontinuální hranice jednoho pokračujícího běhu nebo konverzace, přes kterou lze propojit více modelových kroků a později na ně navázat. ⁽³⁰⁾]

Sezení vymezuje, které interakce patří ke stejnému pokračování; samo o sobě není synonymem pro přepis ani pro aktuální provozní stav. OpenAI Agents SDK umožňuje předat implementaci rozhraní `Session` do `Runner.run`; runner před novým tahem načte uložené konverzační položky, po dokončení přidá nové položky a stejné sezení lze použít i při pokračování přerušného `RunState`. ⁽³⁰⁾

MemorySession a OpenAIConversationsSession. Agents SDK uvádí konkrétní implementace `MemorySession` pro lokální vývoj a `OpenAIConversationsSession` pro Conversations API; obě realizují stejnou hranici pokračování přes rozhraní `Session`. ⁽³⁰⁾

[*Praktický význam:* Stabilní hranice sezení umožňuje navázat další tah nebo obnovený běh na správnou posloupnost interakcí bez směšování nezávislých agentních běhů.]

Přepis (Transcript)

[Přepis je uspořádaný historický záznam toho, co během agentního běhu proběhlo: může obsahovat zprávy, požadavky na nástroje, jejich výsledky a další zaznamenané runtime události. ⁽³⁰⁾]

Přepis odpovídá na otázku, co se stalo. Je historickým důkazem průběhu, nikoli automaticky reprezentací právě platného pracovního stavu ani totožností s aktivním modelovým kontextem. V OpenAI Agents SDK tvoří historii sezení uložené konverzační položky, které runner načítá v pořadí před dalším tahem a doplňuje po dokončení běhu. ⁽³⁰⁾

Historie sezení v Agents SDK. `MemorySession` ukládá konverzační položky v paměti procesu, zatímco jiné implementace stejného rozhraní mohou používat trvalé úložiště. Jde o konkrétní reprezentaci historie, nikoli o tvrzení, že všechny historické události musí být současně aktivním vstupem modelu. ⁽³⁰⁾

[*Praktický význam:* Přepis poskytuje auditovatelnou stopu pro rekonstrukci běhu, diagnostiku chyb a dohledání toho, které pozorování vedlo k pozdějšímu rozhodnutí.]

Stav (State)

[
Stav je persistovaná reprezentace aktuálně platných pracovních skutečností a řídicích údajů potřebných k pokračování vykonávání. ⁽³¹⁾]

Stav se liší od úplného historického přepisu, od aktivního tokenového kontextu modelu i od skutečného stavu externího prostředí. Může z historie odvozovat jen údaje, které jsou pro další krok stále platné, a může uchovávat řídicí informace, jež se do každého modelového vstupu neposílají. OpenAI Agents SDK popisuje `RunState` jako serializovatelný snímek běhu; sandboxová vrstva může v tomto stavu nést také údaje potřebné k opětovnému připojení k pracovnímu prostředí. ^(26, 31)

Obnovení RunState. Přerušný běh Agents SDK lze serializovat a později obnovit místo opakování již provedených kroků; runner tak pokračuje z uložených řídicích údajů a případného navázaného sandboxového stavu. ^(26, 31)

Hranice těchto tří pojmů je proto: **Agentní sezení (Session)** určuje kontinuitu běhu, **Přepis (Transcript)** zachycuje jeho historii a **Stav (State)** uchovává právě platná persistovaná fakta a řídicí údaje.

[*Praktický význam:* Explicitní stav umožňuje obnovit běh podle aktuálně platných údajů bez požadavku, aby celý historický přepis zůstal v modelovém kontextu.]

2.2.2 Prostředí a nástroje

Prostředí agenta (Agent Environment)

[
Prostředí agenta je vnější svět, který Harness agentovi dovoluje pozorovat nebo měnit, například repozitářové soubory, procesy, shell, souborový systém, síťové služby a výstupy testů či sestavení. ⁽²⁶⁾]

Účinky v prostředí existují mimo modelový kontext: změna souboru nebo spuštění procesu není textová inference, ale změna externího stavu, jejíž výsledek musí Harness zprostředkovat zpět jako pozorování. OpenAI Sandbox Agents odděluje definici agenta od živého pracovního prostoru; manifest může nový workspace naplnit repozitářem z GitHubu nebo lokálními soubory a sandboxové schopnosti následně zpřístupní souborový systém a shell. ⁽²⁶⁾

Workspace SandboxAgent. Dokumentace Agents SDK ukazuje agentní workspace inicializovaný repozitářem a následně dostupný přes souborové a shellové schopnosti. Model tak pracuje s reálnými soubory a výstupy procesů, ne s jejich pouhou textovou představou. ⁽²⁶⁾

[*Praktický význam:* Prostředí poskytuje agentovi skutečný pracovní prostor; Harness přitom určuje, které jeho části a operace jsou agentovi vůbec dostupné.]

Nástroje (Tools)

[
Nástroj je externí schopnost nebo programové rozhraní zpřístupněné agentovi, například vyhledávání, čtení souboru, shell, editor nebo API. Nástroj je dostupná schopnost; **Vyvolávání nástrojů (Tool Calling)** je mechanismus, kterým model požádá o její konkrétní použití. ⁽³²⁾]

Harness modelu popíše dostupné nástroje a po zvoleném vyvolání zajistí jejich provedení v příslušném prostředí. OpenAI Agents SDK rozlišuje hostované nástroje, lokálně prováděné nástroje a funkční nástroje; u vestavěných `shell` a `apply_patch` volání požaduje model, ale práci vykonává nakonfigurované prostředí mimo samotnou modelovou odpověď. ⁽³²⁾

Shell a apply_patch. Agents SDK uvádí shell pro spuštění příkazů a apply_patch pro změnu souborů jako konkrétní výkonné nástroje. Model může jejich použití požadovat, zatímco aplikace či sandbox provede skutečný příkaz nebo editaci. ⁽³²⁾

[*Praktický význam:* Nástroje dávají agentovi ověřitelné schopnosti mimo generování textu; jejich rozhraní současně vymezuje, jaké vnější účinky může agent vůbec vyžádat.]

Vyvolávání nástrojů (Tool Calling)

[
Vyvolávání nástrojů je modelové a rozhranové propojení, při kterém model vybere deklarovaný nástroj a vytvoří strukturované argumenty pro konkrétní volání. Samotné vyvolání ještě není provedením nástroje. ⁽³²⁾]

Harness přijme požadavek modelu, ověří jeho argumenty podle rozhraní, směřuje jej na implementaci nástroje, provede operaci mimo model a vrátí výsledek do další inference. OpenAI Agents SDK obaluje lokální funkce jako function tools s JSON Schema a podle schématu jejich argumenty validuje; výchozí agentní smyčka po provedení nástroje vrací jeho výsledek modelu pro další krok. ^(25, 32)

Funkční nástroj v Agents SDK. Quickstart SDK definuje nástroj jako TypeScript funkci se jménem, popisem a schématem Zod; z něj vznikne schéma viditelné modelu a argumenty se před spuštěním funkce validují. ⁽³²⁾

[*Praktický význam:* Oddělení deklarace nástroje, modelového výběru a skutečného provedení dovoluje Harnessu validovat požadavek a kontrolovat účinky předtím, než se projeví v prostředí.]

Spouštění kódu (Code Execution)

[
Spouštění kódu je nástrojová nebo runtime schopnost, která skutečně vykoná program či příkaz a vrátí jeho výstup agentovi jako pozorování. ⁽³³⁾]

Pro softwarovou práci tak lze místo predikce výsledku skutečně spustit test, sestavení, skript, formátovač nebo diagnostický příkaz a získat jeho návratový kód, standardní výstup či chybu. Anthropic Code Execution například provádí Bash příkazy a práci se soubory v serverovém sandboxovaném kontejneru a výsledky vrací modelu ve stejném požadavku. ⁽³³⁾

Anthropic Code Execution. Aktuální rozhraní zpřístupňuje Bash a souborové operace v izolovaném kontejneru; příkaz je vykonán na serveru a model obdrží skutečný výsledek, nikoli odhad toho, co by se při spuštění stalo. ⁽³³⁾

[*Praktický význam:* Skutečné vykonání zvyšuje epistemickou kvalitu agentní práce: další rozhodnutí může vycházet z reálného výsledku testu nebo příkazu místo z pravděpodobnostního odhadu modelu.]

Izolované prostředí (Sandbox)

[
Sandbox je bezpečnostní a izolační hranice, která omezuje prostředky a účinky kódu či nástrojů spuštěných agentem. Není synonymem pro kontejner ani microVM; ty jsou možné technické prostředky jeho realizace. ^(34,35)]

Izolace může kombinovat procesovou nebo virtualizační hranici s omezeními souborového systému, sítě a dalších oprávnění. OpenAI Agents SDK tuto hranici odděluje od konkrétního klienta: `UnixLocalSandboxClient` je určen pro důvěryhodný lokální běh bez OS-level izolace, zatímco `DockerSandboxClient` používá kontejnerovou hranici. Firecracker ukazuje jiný implementační prostředek založený na microVM. ^(34,35)

DockerSandboxClient. Stejný SandboxAgent lze podle dokumentace Agents SDK spustit nad Docker-backed klientem, když je požadována kontejnerová izolace, nebo nad Unix-local klientem pro důvěryhodný lokální vývoj. Rozdíl ukazuje, že sandbox popisuje bezpečnostní hranici, ne jedinou konkrétní virtualizační technologii. ⁽³⁴⁾

[*Praktický význam:* Sandbox omezuje blast radius chybného nebo nežádoucího kroku; Harness má agentovi poskytnout pouze souborový systém, síť a další schopnosti, které daná úloha skutečně potřebuje.]

2.2.3 Rozšíření

Dovednosti (Skills)

[
Dovednost je znovupoužitelný balíček instrukcí a metadat, který může navíc obsahovat podpůrné skripty, reference a další zdroje pro určitou třídu úloh. ⁽³⁶⁾]

Specifikace Agent Skills používá jako vstupní bod `SKILL.md` s YAML frontmatterem a instrukcemi v Markdownu; doplňující obsah může být uložen například v adresářích `scripts/`, `references/` a `assets/` a načítán až podle potřeby. Skill tím přidává znovu-použitelnou pracovní znalost, nikoli sám o sobě novou externí výkonnou schopnost. ⁽³⁶⁾

Skill s podpůrným skriptem. Specifikace počítá s tím, že vedle `SKILL.md` může balíček obsahovat deterministický program v `scripts/` a samostatné referenční materiály. Agent tak může nejprve načíst stručná metadata a detailní instrukce nebo podpůrný kód použít až tehdy, když jsou pro úlohu relevantní. ⁽³⁶⁾

[*Praktický význam:* Skills umožňují verzovat a znovu používat doménové postupy bez opakovaného kopírování celé instrukce do každého zadání a bez zaměňování instrukčního balíčku za samotný nástroj.]

Plugin

[
Plugin je v platformách, které tento mechanismus definují, distribuovatelný balíček rozšíření. V Claude Code může plugin sdružovat například Skills, agenty, Hooks a konfiguraci MCP serverů; tato podoba je specifická pro Claude Code a není univerzální definicí všech agentních pluginů. ⁽³⁷⁾]

Plugin zde představuje především hranici balení a distribuce. Jednotlivé součásti uvnitř něj si zachovávají vlastní význam: Skill nese instrukce a zdroje, Hook reaguje na událost, MCP popisuje integrační protokol a Tool je konkrétní dostupná schopnost. ⁽³⁷⁾

my-first-plugin. Oficiální quickstart Claude Code vytváří adresář `my-first-plugin`, manifest `.claude-plugin/plugin.json` a Skill `skills/hello/SKILL.md`; plugin se pro lokální test načte přepínačem `--plugin-dir`. ⁽³⁷⁾

[*Praktický význam:* Plugin umožňuje distribuovat související rozšíření jako jednu verzovatelnou jednotku, aniž by se jejich jednotlivé mechanismy slévaly do jednoho obecného pojmu.]

Skript

[
Skript je spustitelná deterministická programová logika používaná rozšířením nebo workflow k přesnému a opakovatelnému provedení operace.]

Na rozdíl od nového modelového kroku provádí skript předem definovaný program. Agent Skills specifikace dovoluje balíčku Skill obsahovat podpůrné spustitelné soubory v `scripts/`; Claude Code plugin může obdobně obsahovat `bin/` s executable soubory zpřístupněnými Bash nástroji po dobu aktivace pluginu. ^(36, 37)

Podpůrný program ve Skill. Adresář `scripts/` podle Agent Skills odděluje spustitelnou implementaci od textových instrukcí `SKILL.md`; totéž rozšíření tak může modelu popsat postup a přesný dílčí krok provést deterministickým kódem. ⁽³⁶⁾

[*Praktický význam:* Jestliže má být krok přesný a opakovatelný, vykonání skriptu je vhodnější než požadovat po modelu, aby stejnou mechanickou transformaci pokaždé znovu generoval.]

Hooks

[
Hooks jsou v platformách s událostním rozhraním konfigurované reakce spuštěné při konkrétních událostech životního cyklu. V Claude Code mohou takové reakce spustit deterministický příkaz, HTTP požadavek nebo jinou podporovanou akci nezávisle na tom, zda se model sám rozhodne tuto kontrolu provést. ⁽³⁸⁾]

Událost `PreToolUse` nastává po vytvoření argumentů nástroje, ale před jeho zpracováním, takže Hook může použití povolit, odmítnout nebo vyžádat potvrzení; `PostToolUse` nastává po úspěšném provedení nástroje. Jde o konkrétní lifecycle Claude Code, nikoli o tvrzení, že všechny agentní platformy používají stejné názvy událostí. ⁽³⁸⁾

Lint po změně souboru. Dokumentace Claude Code uvádí `PostToolUse` Hook s matchem pro nástroje `Write|Edit`, který po změně souboru spustí lintovací příkaz. Kontrola se váže na událost runtime, takže její spuštění nemusí model plánovat jako další krok. ⁽³⁸⁾

[*Praktický význam:* Hooks umožňují automaticky připojit povinnou reakci nebo kontrolu ke konkrétní runtime události a oddělit ji od pravděpodobnostního rozhodování modelu.]

MCP

[
Model Context Protocol (MCP) je otevřený protokol pro standardizované propojení AI hostitelů a klientů se servery, které zpřístupňují schopnosti a data. Aktuální specifikace 2026-07-28 definuje hranici mezi hostitelem, klienty a MCP servery; nejde o synonymum pro Tool, Tool Calling ani Plugin. ⁽³⁹⁾]

Server může přes MCP publikovat například nástroje, resources nebo prompts. Protokol standardizuje jejich objevování a vyvolávání přes klientskou hranici, zatímco konkrétní modelový Tool Calling zůstává mechanismem hostitele a Plugin zůstává případným distribučním obalem platformy. ^(39, 40)

Oficiální weather server. TypeScript SDK pro MCP ukazuje server, který registruje nástroj `get-alerts` pro aktivní výstrahy americké National Weather Service. Klient si nástroj vy-

píše a zavolá jej se strukturovaným argumentem státu; SDK argument ověří proti schématu a vrátí výsledek přes protokol. ⁽⁴¹⁾

[*Praktický význam:* MCP umožňuje připojit stejnou serverovou integraci k různým kompatibilním hostitelům bez toho, aby se samotný protokol zaměňoval za konkrétní nástroj nebo rozhodnutí modelu tento nástroj použít.]

.agents/

[
`.agents/` je v aktuálním Codexu jmenný prostor pro agentní přizpůsobení, zejména repozitářové Skills; nejde o umístění souboru AGENTS.md. ⁽⁴²⁾]

Codex hledá repozitářové Skills v `.agents/skills` v cestě projektu a uživatelské Skills v `~/.agents/skills`. Naproti tomu AGENTS.md je samostatný mechanismus instrukcí hledaný v adresářové hierarchii repozitáře. ^(42, 43)

Repozitářový Skill. Umístění `.agents/skills/review/SKILL.md` drží Skill přímo v repozitářovém prostoru Codexu, zatímco kořenový `AGENTS.md` zůstává samostatným souborem instrukcí pro příslušnou část stromu. ^(42, 43)

[*Praktický význam:* `.agents/` umožňuje verzovat znovupoužitelná agentní rozšíření spolu s projektem, aniž by se jejich umístění zaměnilo s hierarchickými instrukcemi AGENTS.md.]

.claude/

[
`.claude/` je projektový nebo uživatelský jmenný prostor Claude Code, ve kterém vedle sebe existuje několik odlišných konfiguračních mechanismů; adresář sám proto není jedním univerzálním typem rozšíření. ^(44, 45)]

Projekt může používat `.claude/CLAUDE.md` pro trvalé instrukce, `.claude/rules/` pro modulární pravidla a `.claude/settings.json` pro sdílená nastavení. Standalone Skills, agenti a Hooks mohou rovněž používat struktury pod `.claude/`, zatímco distribuovatelný Plugin má vlastní kořen a případný manifest `.claude-plugin/plugin.json`. ^(37, 44, 45)

Oddělené projektové soubory. Dokumentace Claude Code ukazuje projekt s `.claude/CLAUDE.md`, samostatnými soubory v `.claude/rules/` a projektovým `.claude/settings.json`; každá cesta má jinou úlohu a vlastní precedence pravidla. ^(44, 45)

[*Praktický význam:* `.claude/` umožňuje verzovat projektové instrukce, pravidla, nastavení a standalone rozšíření blízko repozitáře, přičemž jejich význam zůstává explicitně oddělen.]

Modelová inference poskytuje rozhodnutí nebo výstup pro aktuální krok. Harness k ní přidává kontinuitu, skutečné účinky v externím prostředí a rozšiřitelné běhové rozhraní; společně tím vzniká agentní runtime. Záměrný návrh zadání, kontextu, ověřování a orchestrace tohoto runtime je tématem následující Praktické části.

3 Praktická část

Praktická část převádí teoretické mechanismy do způsobu práce se softwarem. Část 3.1 popisuje a zdůvodňuje postupy Agentického inženýrství; část 3.2 je vyhrazena jejich konkrétní realizaci v systému DarkFactory.

3.1 Agentické inženýrství

[AI ve vývoji softwaru posouvá část práce od přímého psaní k zadávání, delegování a kontrole změn. Agentické inženýrství označuje návrh způsobu, jakým se schopnosti Harnessu skládají, omezují a koordinují tak, aby agent cíleně plnil delší úlohu. ⁽⁷⁾]

Kapitola spojuje způsob formulace práce, řízení změny a nezávislé ověření kvality s návrhem instrukcí, kontextu, řízení chování a orchestrace agentů.

[*Praktický význam:* Agentické inženýrství převádí obecné schopnosti modelu a Harnessu do opakovatelného vývojového procesu s explicitním zadáním, kontrolami, řízeným kontextem a koordinací práce.]

3.1.1 Zadání a způsob práce

Vývoj řízený specifikací (Spec-Driven Development)

[Přístup k AI-asistovanému vývoji, ve kterém explicitní specifikace řídí plánování, implementaci a ověřování změny. ⁽⁴⁶⁾]

Specifikace odděluje požadované chování a omezení od konkrétní implementace a slouží jako společný referenční bod pro člověka i agenta. ⁽⁴⁶⁾

[*Praktický význam:* Specifikace dává agentovi explicitní cíl a akceptační podmínky, podle nichž lze plánovat kroky a ověřovat výsledek.]

Plánování (Planning)

[Převod požadavku na explicitní kroky, závislosti a podmínky ověření před prováděním změn. ⁽⁴⁷⁾]

Plán rozděluje práci na kontrolovatelné části, určuje jejich pořadí a stanovuje podmínky, podle kterých lze posoudit dokončení. ⁽⁴⁷⁾

[*Praktický význam:* Plánování převádí zadání na pořadí kroků a podmínky ověření, podle nichž může agent postupovat a průběžně kontrolovat dokončení.]

Revize (Review)

[Revize je samostatná kontrola změny nebo výstupu proti explicitním požadavkům a kvalitativním kritériím před jeho přijetím.]

GitHub Pull Request review například umožňuje změny komentovat, schválit nebo vrátit s požadavkem na úpravy před sloučením. ⁽⁴⁸⁾

[*Praktický význam:* Revize poskytuje samostatný kontrolní krok proti požadavkům a kvalitativním kritériím, takže nalezené odchylky lze vrátit k opravě před přijetím změny.]

3.1.2 Řízení změny

Správa verzí (Version Control)

[
Systém pro zaznamenávání a porovnávání historie změn souborů v čase. ⁽⁴⁹⁾]
Uložená historie umožňuje identifikovat původ změny, vracet se k předchozím stavům a slučovat samostatně vzniklé změny. ⁽⁴⁹⁾

[*Praktický význam:* Správa verzí umožňuje agentním změnám zůstat dohledatelné, porovnatelné a vratné místo přepisování pracovního stavu bez historie.]

Větev (Branch)

[
Oddělená linie vývoje v systému správy verzí, která ukazuje na vlastní posloupnost commitů. ⁽⁴⁹⁾]

Větev umožňuje izolovat souběžnou změnu od cílové větve a později ji sloučit po kontrole nebo ověření. ^(49, 50)

[*Praktický význam:* Větev izoluje rozpracovanou agentní změnu od hlavní historie a vytváří bezpečný prostor pro testování a revizi před integrací.]

Pull Request

[
Návrh na sloučení změn z jedné větve do jiné, kolem kterého GitHub soustřeďuje revizi, diskusi a automatické kontroly. ⁽⁵¹⁾]

Pull request zpřístupňuje diff navržené změny a její stav před integrací do cílové větve. ⁽⁵¹⁾

[*Praktický význam:* Pull Request vytváří explicitní integrační a revizní hranici, kde lze porovnat změny, spustit kontroly a zaznamenat rozhodnutí před sloučením.]

3.1.3 Kvalita a ověřování

Slop

[
Neformální označení pro nekvalitní digitální obsah, zejména obsah vytvořený umělou inteligencí. ⁽⁵²⁾]

V této práci označuje zejména AI-generovaný software, jehož objem nebo zdánlivá úplnost převyšují jeho ověřenou funkčnost a udržitelnost.

[*Praktický význam:* Riziko nekvalitního generovaného výstupu znamená, že agentní práce potřebuje objektivní kontroly a revizi namísto přijetí výsledku jen proto, že je syntakticky úplný.]

Průběžná integrace (CI)

[
Vývojová praxe, při níž se změny průběžně integrují a automaticky ověřují sestavením, testy a dalšími kontrolami. ⁽⁵³⁾]

CI převádí část podmínek kvality do opakovatelných strojově vyhodnotitelných kontrol spouštěných nad změnami. ⁽⁵³⁾

[*Praktický význam:* CI převádí opakovatelné kontroly změny do automatického signálu, který může agent i člověk použít při rozhodování o dalším kroku.]

Integrační test (Integration Test)

[
Ověření spolupráce více komponent nebo vrstev systému přes jejich rozhraní. ⁽⁴⁷⁾]
Integrační test zachycuje chyby vznikající ve vzájemném propojení částí systému, které izolované testování jednotlivých komponent nemusí odhalit. ⁽⁴⁷⁾

[*Praktický význam:* Integrační test ověřuje spolupráci více částí systému, a proto zachytí chyby, které izolovaná kontrola jednotlivého modulu nebo generovaného souboru neodhalí.]

3.1.4 Instrukce a kontext

Promptové inženýrství (Prompt Engineering)

[
Systematický návrh instrukcí, příkladů a jejich struktury s cílem ovlivnit chování jazykového modelu. ⁽⁵⁴⁾]

Prompt ovlivňuje pravděpodobnostní chování modelu, ale sám nevynucuje technickou bezpečnostní nebo autorizační hranici; pravidla, která musí systém garantovat, patří do **Guardrail** nebo jiné běhové kontroly.

[*Praktický význam:* Promptové inženýrství umožňuje zpřesnit instrukce pro jeden modelový krok; u agentů je však třeba jeho účinek kombinovat se stavem, nástroji a řízením kontextu.]

Systémový prompt (System Prompt)

[
Systémová instrukční vrstva, která vymezuje roli, pravidla a výchozí způsob chování modelu nebo agenta. ⁽⁵⁴⁾]

Systémový prompt poskytuje stabilní instrukční kontext, ale sám o sobě není technickou izolační ani autorizační hranicí. ⁽⁵⁴⁾

[*Praktický význam:* Systémový prompt stanovuje základní instrukce a hranice chování, které Harness přikládá ke každému relevantnímu modelovému kroku.]

AGENTS.md

[

AGENTS.md je mechanismus projektových instrukcí Codexu, který dodává agentovi trvalý repozitářový kontext před zahájením práce. ⁽⁴³⁾]

Codex hledá instrukce od kořene repozitáře směrem k aktuálnímu pracovnímu adresáři, skládá je v tomto pořadí a bližší instrukce tak dostávají vyšší prioritu; na úrovni adresáře může AGENTS.override.md nahradit AGENTS.md. ⁽⁴³⁾

[*Praktický význam:* AGENTS.md umožňuje udržovat repozitářové instrukce přímo u kódu a automaticky je přidávat do kontextu Codexu podle pracovního umístění.]

CLAUDE.md

[
CLAUDE.md je soubor trvalých instrukcí a kontextu, který Claude Code načítá do sezení. ⁽⁴⁴⁾]

Projektové instrukce mohou být v `./CLAUDE.md` nebo `./.claude/CLAUDE.md`, uživatelské v `~/.claude/CLAUDE.md`; soubory nad pracovním adresářem se načítají při spuštění a soubory v podadresářích se mohou načíst až při práci v nich. Jde o kontextové instrukce, nikoli o vynucenou bezpečnostní hranici. ⁽⁴⁴⁾

[*Praktický význam:* CLAUDE.md umožňuje udržovat trvalé projektové nebo uživatelské instrukce, které Claude Code načítá do kontextu sezení.]

Kontextové inženýrství (Context Engineering)

[
Systematický výběr a správa informací, které jsou modelu zpřístupněny v aktivním kontextu během inference. ⁽⁷⁾]

Aktivní kontext může obsahovat instrukce, popisy dostupných nástrojů, externí data a vybranou historii interakce. Jeho obsah je nutné kurátorovat vzhledem k omezené kapacitě a nerovnoměrnému využití dlouhého kontextu. ^(7,23)

[*Praktický význam:* Kontextové inženýrství rozhoduje, které informace mají být v daném kroku modelu skutečně dostupné, což je zásadní pro dlouhé agentní běhy s omezeným rozpočtem.]

Vkládání kontextu (Context Injection)

[
V této práci označuje Context Injection cílené vložení relevantních informací do aktivního kontextu až v okamžiku, kdy jsou potřebné pro aktuální krok. ⁽⁷⁾]

Just-in-time přístup umožňuje mimo modelový kontext uchovávat odkazy nebo trvalý stav a potřebná data načíst nástrojem až během běhu. ⁽⁷⁾

[*Praktický význam:* Vkládání kontextu umožňuje Harnessu doplnit model aktuálními data, instrukce nebo výsledky nástrojů právě v okamžiku, kdy jsou relevantní.]

Kompakce kontextu (Context Compaction)

[

Zmenšení aktivního kontextu nahrazením části historie kratší reprezentací, typicky shrnutím nebo výběrem důležitých informací. ⁽⁷⁾]

Kompakce uvolňuje kapacitu pro další běh, ale příliš agresivní komprese může odstranit detaily, které se později ukážou jako důležité. ^(7,55)

[*Praktický význam:* Kompakce kontextu umožňuje dlouhotrvajícímu agentovi pokračovat přes bezprostřední kontextový rozpočet, za cenu možného ztracení detailů.]

RAG

[
Architektura, ve které systém před generováním vyhledá relevantní informace z externího zdroje a poskytne je modelu jako další kontext. ⁽⁵⁶⁾]

RAG odděluje znalost uloženou v externím korpusu od parametrů modelu a umožňuje vybírat podklady podle aktuálního dotazu. ⁽⁵⁶⁾

[*Praktický význam:* RAG umožňuje před generováním dohledat relevantní externí informace a přidat je do kontextu, takže agent nemusí spoléhat pouze na parametry modelu.]

Prompt Injection

[
Manipulace chování jazykového modelu pomocí instrukcí vložených do vstupu nebo do externího obsahu, který systém následně zpracuje jako kontext. ⁽⁵⁷⁾]

Přímá prompt injection přichází v uživatelském vstupu; nepřímá injection je vložena do externích dat, například webové stránky, e-mailu, dokumentu, repozitáře nebo zdroje RAG. ⁽⁵⁸⁾ Typickým příkladem je životopis obsahující skrytou instrukci, která se pokusí ovlivnit následné hodnocení kandidáta modelem, přestože dokument měl sloužit pouze jako data. ⁽⁵⁷⁾ Důsledky proto závisejí také na oprávněních a nástrojích, které má agent k dispozici. ⁽⁵⁹⁾

[*Praktický význam:* Prompt Injection vyžaduje oddělovat důvěryhodné instrukce od nedůvěryhodného obsahu a omezovat následné nástrojové akce, protože text z prostředí může ovlivnit rozhodování modelu.]

3.1.5 Řízení agentního chování

Cílené smyčky (Goal Loops)

[
V této práci označují řídicí smyčky, které opakují jednání podle explicitního cíle, pozorovaného výsledku a podmínky dalšího pokračování nebo ukončení.]

Na rozdíl od samotného běhového Agent Loopu zahrnuje Goal Loop také zpětnou vazbu k dosažení cíle, ověření výsledku a rozhodnutí, zda pokračovat, změnit postup nebo běh ukončit. ^(9,29)

[*Praktický význam:* Cílená smyčka umožňuje opakovat plánování a ověřování až do splnění explicitní podmínky místo ukončení po prvním přijatelně vypadajícím výstupu.]

Guardrail

[
V této práci označuje Guardrail programově vynucenou kontrolu, která může před pokračováním běhu validovat nebo zablokovat vstup, výstup či použití nástroje. ⁽⁶⁰⁾]

Agentní frameworky mohou guardrails implementovat různými způsoby; zde je důležitá deterministicky vyhodnotitelná hranice oddělená od samotného modelového rozhodnutí. Kontrola může běh zastavit nebo odmítnout konkrétní akci před provedením jejího účinku. ⁽⁶⁰⁾

[*Praktický význam:* Guardrails umožňují deterministickými kontrolami vynucovat omezení, která nemají záviset pouze na tom, zda je model dodrží.]

Člověk ve smyčce (HITL)

[
Uspořádání automatizovaného procesu, ve kterém člověk v určených bodech poskytuje zpětnou vazbu, schválení nebo rozhodnutí. ⁽⁶¹⁾]

HITL ponechává vybraná rozhodnutí člověku místo úplné automatizace. V agentním workflow může být lidský zásah explicitní přechod nebo schvalovací bod, po kterém automatizované provádění pokračuje. ⁽⁶¹⁾

[*Praktický význam:* HITL umožňuje vyžádat lidské rozhodnutí před citlivou nebo nevratnou akcí a tím vložit explicitní schvalovací hranici do jinak automatického běhu.]

3.1.6 Orchestrace agentů

Subagent

[
Specializovaná agentní instance, které jiný agent nebo orchestrátor deleguje vymezenou dílčí úlohu. ⁽⁶²⁾]

Subagent umožňuje oddělit roli, instrukce a pracovní kontext dílčí úlohy od koordinujícího běhu a následně vrátit výsledek zpět nadřazené orchestrace. ^(62, 63)

[*Praktický význam:* Subagent umožňuje oddělit dílčí úlohu do samostatného kontextu a paralelizovat nebo specializovat práci bez zahlcení hlavního agentního vlákna.]

Orchestrátor

[
Koordinační role nebo komponenta, která rozhoduje, kterému specializovanému agentovi předat dílčí práci a jak jeho výsledek začlenit do pokračujícího běhu. ⁽⁶²⁾]

Centralizovaný orchestrátor zůstává vlastníkem hlavního workflow a může specialisty volat jako omezené pracovní jednotky, případně jejich práci kombinovat nebo spouštět paralelně. ^(9, 62)

[*Praktický význam:* Orchestrátor rozděluje práci, spouští dílčí vykonavatele a skládá jejich výsledky, čímž umožňuje koordinovat více agentních větví jako jeden proces.]

Předání řízení (Handoff)

[
Vzor koordinace, při kterém aktivní agent předá další řízení specializovanému agentovi.
(62)]

Handoff se liší od centralizované orchestrace tím, že specialista není pouze zavolán jako dílčí pracovní jednotka a vrácen orchestrátoru, ale přebírá aktivní pokračování interakce nebo úlohy. (62)

[*Praktický význam:* Handoff předává odpovědnost i potřebný kontext jinému agentovi nebo roli, takže další krok nemusí pokračovat ve stejném agentním vlákne.]

Graf pracovního postupu (Workflow Graph)

[
Explicitní grafová reprezentace vícefázového workflow, v níž uzly představují dílčí práci a hrany určují přechody nebo závislosti mezi kroky. (63)]

Workflow Graph může vyjádřit sekvenční, podmíněné i paralelní větvení a může koordinovat více agentů. Pokud hrany vyjadřují pouze acyklické závislosti, může mít podobu DAG; workflow s návraty nebo opakováním však obecně DAG být nemusí. (9, 63)

[*Praktický význam:* Workflow Graph explicitně zachycuje návaznosti, větvení, paralelismus a smyčky, takže složitější agentní proces lze řídit jako strukturovaný tok práce.]

Swarm

[
Swarm označuje dynamicky koordinované paralelní provádění úlohy více subagenty pod orchestrujícím agentem nebo systémem; nejde o univerzální formální standard.]

Kimi K2.5 Agent Swarm je konkrétní realizace, v níž orchestrátor dynamicky vytváří a koordinuje paralelní subagenty bez předem definovaných rolí nebo ručně napsaného workflow. (64)

[*Praktický význam:* Swarm umožňuje dynamicky rozdělit rozsáhlou úlohu mezi více paralelních subagentů, pokud zadání přirozeně obsahuje nezávislé části práce.]

Delegování práce AI nezmenšuje potřebu softwarově-inženýrských kontrol; přesouvá jejich význam k explicitnímu zadání, sledovatelným změnám, řízenému kontextu a strojově ověřitelné zpětné vazbě. Tyto principy vytvářejí základ pro konkrétní realizaci v systému DarkFactory.

3.2 DarkFactory

4 Výsledky a diskuse

[
Vyhodnocení odděluje důkazy o jednotlivých mechanismech od důkazů o jejich spolupráci na úrovni systému a od přenosu na konkrétní cílové repozitáře.]

Metodika těchto důkazů je stanovena v části 1.5; zde se už neopakuje. Vyhodnocení používá reprodukovatelný evidence snapshot DarkFactory na commitu `e9c10221b40589512d262a0edb95f709b923150c` a odpovídající CI run `35616745304`. Snapshot slouží pouze jako pevný referenční bod pro výsledky. ^(65, 66)

4.1 Ověření mechanismů

[
Referenční CI run dokončil všech 15 jobů úspěšně a hlavní Bun testovací sada vykázala 670 úspěšných testů ve 102 souborech bez selhání. ⁽⁶⁶⁾]

Samostatné testy ověřují mimo jiné persistenci Run State, detekci neaktuálního Planningu, zachování historie při provider failoveru, odvozování výsledku z pracovního stromu, kontrolu scope a recovery provenance. ⁽⁶⁵⁾ Tato evidence podporuje konkrétní mechanismy; sama o sobě neprokazuje úspěšnost celého produkčního životního cyklu.

4.2 Ověření systému

[
Integrační testy dokazují spolupráci hlavních runtime mechanismů, ale evidence uza-
vřená 21. září 2026 neobsahuje jeden reprodukovatelný živý průchod celým produkčním
životním cyklem změny.]

Testy pokrývají workflow přechody, review/fix iterace, persistovaný stav, GitHub udá-
losti, required-check gate, nástrojové účinky, failover a recovery. ⁽⁶⁵⁾ Plný df-only průchod
od schválení Planningu po merge a následnou rekonsiliaci zůstal zároveň otevřenou accep-
tance položkou Requestu 359, a proto není v této práci považován za prokázaný. ⁽⁶⁷⁾

4.3 Ověření na repozitářích

[
Fleet-level ověření rozlišuje aktivní cílové repozitáře od historických nebo archivova-
ných položek a hodnotí pouze konkrétní commity s dohledatelnými workflow výsledky.]

Repozitář `omnis` byl ověřen na commitu `a53660a1c0c6619f94768e5d520405052fb03df6`. Jeho pipeline run `34708160162` dokončil
úspěšně mimo jiné web, paper a docs jobs; samostatné deploy-docs a release joby na stejném
commitu byly rovněž úspěšné. ^(68, 69)

Repozitář `ChessWithQuests` byl ověřen na commitu `50a50797f29c2a636d973981993191a65df3d131`. Pipeline run `34708180783` dokončil úspěšně paper, web a docs jobs a na stejném commitu uspěly také verify-docs, deploy-docs a release kontroly. ^(70,71)

Původně uváděné repozitáře `template-OdbornaPrace` a `OdbornaPrace-mono` jsou v době evaluace archivované a nejsou proto považovány za aktivní fleet cíle. ^(72,73)

Aktivním repozitářem práce je `DarkFactory-Paper`, který nahrazuje starší označení `OdbornaPrace-paper`. Snapshot `5bc04974f9aed0f55295389154124a09059ff35e` prošel úspěšně CI, Deploy Documentation i Release workflow. ⁽⁷⁴⁻⁷⁷⁾

Aktivní fleet evidence tedy v této fázi pokrývá DarkFactory, omnis, ChessWithQuests a DarkFactory-Paper. Dvě původní fleet položky jsou archivované; úplné 361 fleet acceptance proto nelze z těchto výsledků považovat za uzavřené.

4.4 Výzkumné otázky

[
Odpovědi jsou omezeny na vlastnosti doložené implementací, automatickými testy, CI a dostupnými provozními artefakty.]

01. Řízenou autonomii podporuje oddělení schváleného plánu, implementace, deterministického ověření, review/fix smyčky, Final Alignment a merge autorizace. Jejich dílčí kontrakty jsou implementované a testované, ale evidence neobsahuje jeden úplný živý produkční průchod. ⁽⁶⁵⁻⁶⁷⁾

02. Přerušný nebo neproduktivní běh lze omezovat pomocí budgetů, klasifikace chyb a failoveru a obnovovat nad persistovaným stavem s kontrolou provenance. Testy tyto mechanismy podporují, nikoli však živou crash/resume idempotenci nad celým produkčním workflow. ^(65,67)

03. Trvalý pracovní stav je oddělen od context window: Run State, Session a Transcript mohou existovat mimo aktuální inferenční vstup a aktivní kontext z nich vybírá pouze informace potřebné pro další krok. Tato architektura podporuje pokračování přes hranice modelových běhů, ale nedokazuje nulovou informační ztrátu pro libovolně dlouhou úlohu. ⁽⁶⁵⁾

4.5 Diskuse a omezení

[
Dostupná evidence podporuje realizaci a testované vlastnosti jednotlivých mechanismů, nikoli obecné tvrzení o úplné autonomii nebo výkonnostní převaze DarkFactory.]

Práce neprovádí statistický benchmark úspěšnosti, ceny, latence ani četnosti zacyklení na reprezentativním souboru úloh. Evidence také neobsahuje jeden živý plný Request lifecycle ani úplnou acceptance původně plánované fleety. ⁽⁶⁷⁾ Silnou stránkou evaluace je naopak přímá vazba konkrétních tvrzení na implementační a testovací důkazy, například pozorovaný stav pracovního stromu nebo recovery provenance. ⁽⁶⁵⁾

Evaluace prokazuje implementaci a automatické ověření klíčových mechanismů Dark-Factory a současně vymezuje chybějící systémové důkazy. Hlavní cíl je proto hodnocen na úrovni navržené architektury a reprodukovatelně testovaných mechanismů; neprovedený plný živý lifecycle zůstává explicitním omezením závěrů. ⁽⁶⁷⁾

5 Závěr

[
Práce ukazuje, že delegování softwarové práce agentní AI vyžaduje více než samotnou schopnost jazykového modelu generovat kód: rozhodující jsou mechanismy, které mimo model udržují stav, zprostředkovávají účinky a poskytují ověřitelnou zpětnou vazbu.]

DarkFactory demonstruje konkrétní realizaci tohoto přístupu a dostupná evaluace podporuje funkčnost klíčových mechanismů i jejich vybraných integrací. Výzkumné otázky proto lze uzavřít na úrovni řízené autonomie, obnovitelnosti běhu a oddělení trvalého stavu od omezeného modelového kontextu.

Rozsah závěru je omezen provedenými důkazy: práce neprokazuje obecnou převahu DarkFactory ani jeden úplný živý průchod celým produkčním životním cyklem změny. Tato hranice je součástí výsledku evaluace, nikoli nahrazena silnějším tvrzením. ⁽⁶⁷⁾

Seznam zdrojů

1. HILLEBRANDT, Finn. How Many People Use AI? Facts \& Figures 2026. Online. 2026. [Accessed 22 září 2026]. Available from: <https://www.gradually.ai/en/how-many-people-use-ai/>Published 2026-08-23
2. KARPATY, Andrej. A new kind of coding I call vibe coding. Online. 2025. [Accessed 20 září 2026]. Available from: <https://twitter.com/karpathy/status/1886192184808149383>
3. WILLISON, Simon. A quote from Andrej Karpathy. Online. 2025. [Accessed 20 září 2026]. Available from: <https://simonwillison.net/2025/Feb/6/andrej-karpathy/>
4. KRAVETS, David. A semantic history: How the term 'vibe coding' went from a tweet to prod. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.coderabbit.ai/blog/a-semantic-history-how-the-term-vibe-coding-went-from-a-tweet-to-prod>
5. BARRY, Alexander. The ECI frontier has advanced by 14 points per year since the introduction of reasoning models. Online. 2026. [Accessed 22 září 2026]. Available from: <https://epoch.ai/data-insights/eci-frontier-trend>Published 2026-09-01; model ECI scores and trend data version 2026-09-01
6. ARTIFICIAL ANALYSIS. Artificial Analysis Intelligence Index v4.3.2. Online. 2026. [Accessed 22 září 2026]. Available from: <https://artificialanalysis.ai/evaluations/artificial-analysis-intelligence-index>Version 4.3.2 published 2026-09-07; point-in-time snapshot observed 2026-09-22
7. ANTHROPIC. Effective context engineering for AI agents. Online. 2025. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
8. ANTHROPIC. Harness design for long-running application development. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
9. ANTHROPIC. Building effective agents. Online. 2024. Available from: <https://www.anthropic.com/research/building-effective-agents>
10. ANTHROPIC. Scaling Managed Agents: Decoupling the brain from the hands. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/managed-agents>
11. BROWN, Tom B., MANN, Benjamin, RYDER, Nick, SUBBIAH, Melanie, KAPLAN, Jared, DHARIWAL, Prafulla, NEELAKANTAN, Arvind, SHYAM, Pranav, SASTRY, Girish, ASKELL, Amanda, HERBERT-VOSS, Ariel, KRUEGER, Gretchen, HENIGHAN, Tom, CHILD, Rewon, RAMESH, Aditya, ZIEGLER, Daniel M., WU, Jeffrey, WINTER, Clemens, HESSE, Christopher, CHEN, Mark, SIGLER, Eric, LITWIN, Mateusz, GRAY, Scott, CHES, Benjamin, CLARK, Jack, BERNER, Christopher, MCCANDLISH, Sam, RADFORD, Alec,

- SUTSKEVER, Ilya a AMODEI, Dario. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33. Available from: <https://arxiv.org/abs/2005.14165>
12. HEVNER, Alan R., MARCH, Salvatore T., PARK, Jinsoo a RAM, Sudha. Design Science in Information Systems Research. *MIS Quarterly*. 2004. Vol. 28, no. 1.
 13. PEFFERS, Ken, TUUNANEN, Tuure, ROTHENBERGER, Marcus A. a CHATTERJEE, Samir. A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*. 2007. Vol. 24, no. 3, p. 45–77. DOI [10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302).
 14. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
 15. SENNRICH, Rico, HADDOW, Barry a BIRCH, Alexandra. Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Online. 2016. P. 1715–1725. Available from: <https://aclanthology.org/P16-1162/>
 16. MIKOLOV, Tomas, CHEN, Kai, CORRADO, Greg a DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. Online. 2013. Available from: <https://arxiv.org/abs/1301.3781>
 17. MIKOLOV, Tomas, YIH, Wen-tau a ZWEIG, Geoffrey. Linguistic Regularities in Continuous Space Word Representations. In : *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Online. 2013. p. 746–751. Available from: <https://aclanthology.org/N13-1090/>
 18. MIKOLOV, Tomas, SUTSKEVER, Ilya, CHEN, Kai, CORRADO, Greg S. a DEAN, Jeffrey. Distributed Representations of Words and Phrases and their Compositionality. *Advances in Neural Information Processing Systems*. Online. 2013. Vol. 26. Available from: <https://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality>
 19. OPENAI. Models | OpenAI Agents SDK. Online. 2026. [Accessed 22 září 2026]. Available from: <https://openai.github.io/openai-agents-python/models/>
 20. VLLM PROJECT. vLLM: A High-Throughput and Memory-Efficient Inference Engine for LLMs. Online. 2026. [Accessed 22 září 2026]. Available from: <https://docs.vllm.ai/en/stable/api/vllm/>
 21. KWON, Woosuk, LI, Zhuohan, ZHUANG, Siyuan, SHENG, Ying, ZHENG, Lianmin, YU, Cody Hao, GONZALEZ, Joseph E., ZHANG, Hao a STOICA, Ion. Efficient Memory

- Management for Large Language Model Serving with PagedAttention. *arXiv preprint arXiv:2309.06180*. Online. 2023. Available from: <https://arxiv.org/abs/2309.06180>
22. OPENAI. Create a model response | OpenAI API Reference. Online. 2026. [Accessed 22 září 2026]. Available from: <https://developers.openai.com/api/reference/cli/resources/responses/methods/create>
 23. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
 24. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
 25. OPENAI. OpenAI Agents SDK TypeScript. Online. 2026. [Accessed 22 září 2026]. Available from: <https://openai.github.io/openai-agents-js/>
 26. OPENAI. Sandbox agents: Concepts | OpenAI Agents SDK. Online. 2026. [Accessed 22 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/sandbox-agents/concepts/>
 27. ANTHROPIC. Enabling Claude Code to work more autonomously. Online. 2025. [Accessed 22 září 2026]. Available from: <https://www.anthropic.com/news/enabling-claude-code-to-work-more-autonomously>
 28. GOOGLE. Getting Started with Antigravity IDE. Online. 2026. [Accessed 22 září 2026]. Available from: <https://codelabs.developers.google.com/getting-started-agy-ide>
 29. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
 30. OPENAI. Sessions | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/sessions/>
 31. OPENAI. RunState | OpenAI Agents SDK. Online. 2026. [Accessed 22 září 2026]. Available from: <https://openai.github.io/openai-agents-js/openai/agents-core/classes/runstate/>
 32. OPENAI. Tools | OpenAI Agents SDK. Online. 2026. [Accessed 22 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/tools/>
 33. ANTHROPIC. Code execution tool. Online. 2026. [Accessed 20 září 2026]. Available from: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/code-execution-tool>

34. OPENAI. Sandbox clients | OpenAI Agents SDK. Online. 2026. [Accessed 22 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/sandbox-agents/clients/>
35. AGACHE, Alexandru, DEACONESCU, Razvan, IORDACHE, Mihai, LUPU, Alexandra, RADU, Vlad, BUGA, Cezar, BALAN, Catalin a FLORESCU, Andreea. Firecracker: Lightweight Virtualization for Serverless Applications. In : *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Online. 2020. p. 419–434. Available from: <https://www.usenix.org/conference/nsdi20/presentation/agache>
36. AGENT SKILLS. Agent Skills Specification. Online. 2026. [Accessed 20 září 2026]. Available from: <https://agentskills.io/specification>
37. ANTHROPIC. Create plugins | Claude Code Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/plugins>
38. ANTHROPIC. Hooks reference | Claude Code Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/hooks>
39. MODEL CONTEXT PROTOCOL. Architecture — Model Context Protocol specification 2026-07-28. Online. 2026. [Accessed 22 září 2026]. Available from: <https://modelcontextprotocol.io/specification/2026-07-28/architecture>
40. MODEL CONTEXT PROTOCOL. Tools — Model Context Protocol specification 2026-07-28. Online. 2026. [Accessed 22 září 2026]. Available from: <https://modelcontextprotocol.io/specification/2026-07-28/server/tools>
41. MODEL CONTEXT PROTOCOL. Build your first server | MCP TypeScript SDK. Online. 2026. [Accessed 22 září 2026]. Available from: <https://ts.sdk.modelcontextprotocol.io/v2/get-started/first-server>
42. OPENAI. Customization overview. Online. 2026. [Accessed 22 září 2026]. Available from: <https://developers.openai.com/docs/customization/overview>
43. OPENAI. Custom instructions with AGENTS.md. Online. 2026. [Accessed 22 září 2026]. Available from: <https://developers.openai.com/docs/agent-configuration/agents-md>
44. ANTHROPIC. How Claude remembers your project | Claude Code Docs. Online. 2026. [Accessed 22 září 2026]. Available from: <https://code.claude.com/docs/en/memory>
45. ANTHROPIC. Settings files and precedence | Claude Code Docs. Online. 2026. [Accessed 22 září 2026]. Available from: <https://code.claude.com/docs/en/settings>
46. BÖCKELER, Birgitta. Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessel. Online. 2025. [Accessed 21 září 2026]. Available from: <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
47. SOMMERVILLE, Ian. *Software Engineering*. 10. Pearson, 2016. ISBN 978-0-13-394303-0.
48. GITHUB. Pull request reviews | GitHub Docs. Online. 2026. [Accessed 22 září 2026]. Available from: <https://docs.github.com/en/pull-requests/reference/pull-request-reviews>

49. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. IS-BN 978-1-4842-0076-6.
50. GITHUB. Branches | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/reference/branches>
51. GITHUB. About pull requests | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>
52. CAMBRIDGE UNIVERSITY PRESS. AI slop. Online. 2026. [Accessed 20 září 2026]. Available from: <https://dictionary.cambridge.org/dictionary/english/ai-slop>
53. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
54. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
55. JIANG, Huiqiang, WU, Qianhui, LIN, Chin-Yew, YANG, Yuqing a QIU, Lili. LLMLingua: Compressing Context for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736*. Online. 2023. Available from: <https://arxiv.org/abs/2310.05736>
56. LEWIS, Patrick, PEREZ, Ethan, PIKTUS, Aleksandra, PETRONI, Fabio, KARPUKHIN, Vladimir, GOYAL, Naman, KÜTTLER, Heinrich, LEWIS, Mike, YIH, Wen-tau, ROCKTÄSCHEL, Tim, RIEDEL, Sebastian a KIELA, Douwe. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33, p. 9459–9474. Available from: <https://arxiv.org/abs/2005.11401>
57. OWASP GENAI SECURITY PROJECT. LLM01:2025 Prompt Injection. Online. 2025. [Accessed 21 září 2026]. Available from: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
58. OWASP FOUNDATION. LLM Prompt Injection Prevention Cheat Sheet. Online. 2026. [Accessed 21 září 2026]. Available from: https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
59. OPENAI. Understanding prompt injections. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.com/safety/prompt-injections/>
60. OPENAI. Guardrails | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/guardrails/>
61. MOSQUEIRA-REY, Eduardo, HERNÁNDEZ-PEREIRA, Elena, ALONSO-RÍOS, David, BOBES-BASCARÁN, José a FERNÁNDEZ-LEAL, Ángel. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*. Online. 2023. Vol. 56, no. 4, p. 3005–3054. Available from: <https://doi.org/10.1007/s10462-022-10246-w>

62. OPENAI. Agent orchestration | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: https://openai.github.io/openai-agents-python/multi_agent/
63. WU, Qingyun, BANSAL, Gagan, ZHANG, Jieyu, WU, Yiran, LI, Beibin, ZHU, Erkang, JIANG, Li, ZHANG, Xiaoyun, ZHANG, Chi, LIU, Jue, AWADALLAH, Ahmed Hassan, WHITE, Ryen W., BURGER, Doug a WANG, Chi. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*. Online. 2023. Available from: <https://arxiv.org/abs/2308.08155>
64. MOONSHOT AI. Kimi K2.5: Visual Agentic Intelligence. Online. 2026. [Accessed 22 září 2026]. Available from: <https://www.kimi.com/en/blog/kimi-k2-5>
65. MARIUS, Patrik. DarkFactory commit e9c10221. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/commit/e9c10221b40589512d262a0edb95f709b923150c>
66. GITHUB ACTIONS. DarkFactory CI run 35616745304. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/actions/runs/35616745304>
67. MARIUS, Patrik. Request #359: finish the df production engine and retire Python orchestration. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/issues/359>
68. MARIUS, Patrik. omnis commit a53660a1. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/omnis/commit/a53660a1c0c6619f94768e5d520405052fb03df6>
69. GITHUB ACTIONS. omnis pipeline run 34708160162. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/omnis/actions/runs/34708160162>
70. MARIUS, Patrik. ChessWithQuests commit 50a50797. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/ChessWithQuests/commit/50a50797f29c2a636d973981993191a65df3d131>
71. GITHUB ACTIONS. ChessWithQuests pipeline run 34708180783. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/ChessWithQuests/actions/runs/34708180783>
72. MARIUS, Patrik. template-OdbornaPrace repository. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/template-OdbornaPrace>
73. MARIUS, Patrik. OdbornaPrace-mono repository. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/OdbornaPrace-mono>
74. MARIUS, Patrik. DarkFactory-Paper commit 5bc04974. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/commit/5bc04974f9aed0f55295389154124a09059ff35e>

75. GITHUB ACTIONS. DarkFactory-Paper CI run 35617820423. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820423>
76. GITHUB ACTIONS. DarkFactory-Paper Deploy Documentation run 35617820271. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820271>
77. GITHUB ACTIONS. DarkFactory-Paper Release run 35617820286. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820286>
78. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
79. ANTHROPIC. Extend Claude with skills | Claude Code Docs. Online. 2026. [Accessed 22 září 2026]. Available from: <https://code.claude.com/docs/en/skills>

Seznam obrázků a tabulek

- Obrázek 1 Odhad rozdělení světové populace podle nejpokročilejší používané kategorie generativní AI v srpnu 2026. Každý z 2 500 bodů představuje přibližně 3,3 milionu lidí; kategorie jsou vzájemně výlučné. Hodnota pro pravidelné uživatele AI coding agents je redakční, deduplikovaný odhad Gradually (25–35 milionů), přičemž graf používá střed 30 milionů, přibližně 0,36 % populace. ⁽¹⁾ 8
- Obrázek 2 Původní tweet Andreje Karpathyho o Vibe Coding. ^(2,4) 9
- Obrázek 3 Frontier pozorování a lineární trendy Epoch Capabilities Index (ECI) od přechodu k reasoning modelům v září 2024. Epoch uvádí přibližně 14 ECI bodů za rok pro reasoning frontier a přibližně 6 bodů za rok pro non-reasoning frontier; zdrojová data jsou ve verzi z 1. 9. 2026. Pro čitelnost v tisku nejsou zobrazeny 90% predikční intervaly. ⁽⁵⁾ 10
- Tabulka 1 Bodový snímek pěti nejvýše skórujících odlišných základních modelů podle Artificial Analysis Intelligence Index v4.3.2, zveřejněného 7. 9. 2026; hodnoty byly ověřeny 22. 9. 2026. U každého základního modelu je ponechána jeho nejvýše skórující vyhodnocená konfigurace. ⁽⁶⁾ 10
- Obrázek 4 Ilustrativní 2D projekce vztahu král – muž + žena $\approx$ královna. Osy Pohlaví a Královský status jsou vysvětlující projekcí pro názornost, nikoli doslovnými naučenými dimenzemi nebo produkčními souřadnicemi embeddingového prostoru. 16
- Obrázek 5 Pedagogická 3D projekce dvou relačních rodin. Reálné embeddingové prostory jsou vysokodimenzionální; osy Pohlaví, Královský status a Další sémantická dimenze jsou pouze vysvětlující projekční pomůcky a nepředstavují doslovné produkční souřadnice embeddingu. 17
- Obrázek 6 Terminálové rozhraní Claude Code jako příklad uživatelské vrstvy agentního Harnessu: model je zpřístupněn uvnitř pokračujícího běhu s viditelným stavem práce a nástrojovými interakcemi. Snímek pochází z oficiálního materiálu Anthropic. ⁽²⁷⁾ 20
- Obrázek 7 Google Antigravity IDE jako příklad integrovaného agentního prostředí: editor, terminál a agentní panel tvoří rozhraní k pracovnímu prostoru, ve kterém může agent pozorovat a měnit software. Snímek pochází z oficiálního Google Colab. ⁽²⁸⁾ 21
- Obrázek 8 Schéma agentní smyčky inspirované interakcí ReAct: model volí akci, Harness ji předá nástroji nebo prostředí, pozorování výsledku se vrací modelu a cyklus

může skončit výsledkem. ReAct je zde zdrojem mechanismu, nikoli tvrzením,
že každý agentní systém používá totožnou architekturu. ⁽²⁹⁾ 22

Seznam příloh

A Encyklopedie a rejstřík pojmů	51
---------------------------------------	----

A Encyklopedie a rejstřík pojmů

Abecední přehled klíčových pojmů použitých v práci. Názvy a definice jsou odvozeny přímo z kanonických sémantických položek.

.agents/

`.agents/` je repozitářový nebo uživatelský jmenný prostor Codexu pro znovupoužitelná agentní rozšíření, zejména Skills. ⁽⁴²⁾

.claude/

`.claude/` je projektový nebo uživatelský jmenný prostor Claude Code pro instrukce, pravidla, nastavení a rozšíření. ^(44, 45)

Agentní sezení (Session)

Persistovaná jednotka, která vymezuje jeden souvislý agentní běh a umožňuje jeho pozdější pokračování. ⁽³⁰⁾

Agentní smyčka (Agent Loop)

Iterativní cyklus, v němž model vyhodnotí stav, zvolí akci, harness ji provede a výsledek vrátí do další iterace. ⁽²⁹⁾

AGENTS.md

AGENTS.md je mechanismus projektových instrukcí Codexu, který dodává agentovi trvalý repozitářový kontext před zahájením práce. ⁽⁴³⁾

CLAUDE.md

CLAUDE.md je soubor trvalých instrukcí a kontextu, který Claude Code načítá do sezení. ⁽⁴⁴⁾

Cílené smyčky (Goal Loops)

V této práci označují řídicí smyčky, které opakují jednání podle explicitního cíle, pozorovaného výsledku a podmínky dalšího pokračování nebo ukončení.

Degradace kontextu (Context Rot)

Degradace kontextu (Context Rot) označuje praktický pokles spolehlivosti, s níž model dokáže využívat relevantní informace při růstu délky, informačního zatížení nebo nevýhodném umístění informace v aktivním kontextu. ⁽²³⁾

Dovednosti (Skills)

Znovupoužitelný balíček instrukcí a volitelných zdrojů, který se načítá pro úlohy odpovídající jeho účelu. ⁽³⁶⁾

Graf pracovního postupu (Workflow Graph)

Explicitní grafová reprezentace vícefázového workflow, v níž uzly představují dílčí práci a hrany určují přechody nebo závislosti mezi kroky. ⁽⁶³⁾

Guardrail

V této práci označuje Guardrail programově vynucenou kontrolu, která může před pokračováním běhu validovat nebo zablokovat vstup, výstup či použití nástroje. ⁽⁶⁰⁾

Harness

Harness je běhová vrstva kolem modelové inference, která drží stav, opakuje agentní smyčku a propojuje model s nástroji a prostředím. ⁽¹⁰⁾

Hooks

Konfigurované reakce spouštěné při určených událostech životního cyklu agentního prostředí. ⁽³⁸⁾

Inferenční engine (Inference Engine)

Inferenční engine je běhová vrstva, která načítá model a skutečně provádí jeho dopředné výpočty a autoregresivní generování nad vstupními tokeny. ⁽²⁰⁾

Integrační test (Integration Test)

Ověření spolupráce více komponent nebo vrstev systému přes jejich rozhraní. ⁽⁴⁷⁾

Izolované prostředí (Sandbox)

Oddělené běhové prostředí, ve kterém agent může spouštět kód nebo měnit pracovní soubory bez přímého přístupu ke všem prostředkům hostitelského systému. ⁽¹⁰⁾

Kompakce kontextu (Context Compaction)

Zmenšení aktivního kontextu nahrazením části historie kratší reprezentací, typicky shrnutím nebo výběrem důležitých informací. ⁽⁷⁾

Kontextové inženýrství (Context Engineering)

Systematický výběr a správa informací, které jsou modelu zpřístupněny v aktivním kontextu během inference. ⁽⁷⁾

Kontextové okno (Context Window)

Kontextové okno je konečný rozsah tokenové sekvence, kterou model může mít v daném inferenčním běhu současně k dispozici jako aktivní vstup. ⁽²³⁾

MCP

Otevřený protokol pro standardizované propojení AI aplikací s externími nástroji a datovými zdroji. ⁽³⁹⁾

Mezipaměť klíčů a hodnot (KV Cache)

KV cache je runtime mezipaměť dříve vypočtených klíčů a hodnot pozornostních vrstev pro tokeny již zpracovaného prefixu, které lze znovu použít při autoregresivním dekodování dalších tokenů. ⁽²⁴⁾

Nástroje (Tools)

Rozhraní, kterým agent vyvolává operace mimo samotnou textovou inferenci, například čtení dat, volání API nebo změnu stavu systému. ⁽⁹⁾

Plánování (Planning)

Převod požadavku na explicitní kroky, závislosti a podmínky ověření před prováděním změn. ⁽⁴⁷⁾

Poskytovatel modelu (Model Provider)

Poskytovatel modelu je externí služba nebo programové rozhraní, přes které runtime vybírá a volá konkrétní model; provider může mapovat abstraktní jméno modelu na vlastní implementaci modelového API. ⁽¹⁹⁾

Prompt Injection

Manipulace chování jazykového modelu pomocí instrukcí vložených do vstupu nebo do externího obsahu, který systém následně zpracuje jako kontext. ⁽⁵⁷⁾

Promptové inženýrství (Prompt Engineering)

Systematický návrh instrukcí, příkladů a jejich struktury s cílem ovlivnit chování jazykového modelu. ⁽⁵⁴⁾

Prostředí agenta (Agent Environment)

Vnější prostředí, které agent prostřednictvím harnessu pozoruje a mění, například pracovní soubory, procesy, síťové služby a další systémové prostředky. ⁽¹⁰⁾

Průběžná integrace (CI)

Vývojová praxe, při níž se změny průběžně integrují a automaticky ověřují sestavením, testy a dalšími kontrolami. ⁽⁵³⁾

Pull Request

Návrh na sloučení změn z jedné větve do jiné, kolem kterého GitHub soustřeďuje revizi, diskusi a automatické kontroly. ⁽⁵¹⁾

Předání řízení (Handoff)

Vzor koordinace, při kterém aktivní agent předá další řízení specializovanému agentovi.

(62)

Přepis (Transcript)

Uspořádaný historický záznam událostí vzniklých během *Agentní sezení (Session)*, například zpráv, akcí a výsledků nástrojů. ⁽¹⁰⁾

RAG

Architektura, ve které systém před generováním vyhledá relevantní informace z externího zdroje a poskytne je modelu jako další kontext. ⁽⁵⁶⁾

Revize (Review)

Revize je samostatná kontrola změny nebo výstupu proti explicitním požadavkům a kvalitativním kritériím před jeho přijetím.

Slop

Neformální označení pro nekvalitní digitální obsah, zejména obsah vytvořený umělou inteligencí. ⁽⁵²⁾

Spouštění kódu (Code Execution)

Nástrojová schopnost umožňující vykonat program nebo příkaz a vrátit jeho skutečný výstup modelu. ⁽³³⁾

Správa verzí (Version Control)

Systém pro zaznamenávání a porovnávání historie změn souborů v čase. ⁽⁴⁹⁾

Stav (State)

Persistovaná reprezentace aktuálně platných pracovních skutečností a řídicích údajů běhu. ⁽¹⁰⁾

Subagent

Specializovaná agentní instance, které jiný agent nebo orchestrátor deleguje vymezenou dílčí úlohu. ⁽⁶²⁾

Swarm

Swarm označuje dynamicky koordinované paralelní provádění úlohy více subagenty pod orchestrujícím agentem nebo systémem; nejde o univerzální formální standard.

Systémový prompt (System Prompt)

Systémová instrukční vrstva, která vymezuje roli, pravidla a výchozí způsob chování modelu nebo agenta. ⁽⁵⁴⁾

Teplota (Temperature)

Teplota je parametr vzorkování, který u rozhraní, jež jej podporují, mění koncentraci pravděpodobnostního výběru dalších tokenů a tím ovlivňuje variabilitu generovaného výstupu. ⁽²²⁾

Token

Token je diskrétní jednotka sekvence identifikovaná položkou slovníku tokenizéru; podle tokenizační metody může odpovídat celému slovu, části slova nebo jinému textovému fragmentu. ⁽¹⁵⁾

Transformer

Transformer je architektura neuronové sítě, která zpracovává vztahy v sekvenci pomocí mechanismů pozornosti místo rekurence či konvoluce jako základního mechanismu pro přenos informace mezi pozicemi. ⁽¹⁴⁾

Vektorová reprezentace (Embedding)

Embedding je spojitá vícerozměrná vektorová reprezentace diskrétního prvku, v níž se naučené geometrické vztahy mohou využít k zachycení podobnosti a dalších vztahů mezi reprezentovanými objekty. ⁽¹⁶⁾

Velký jazykový model (LLM)

Velký jazykový model (LLM) je parametrický model pravděpodobnostního rozdělení nad posloupnostmi tokenů; autoregresivní LLM generuje pokračování postupným odhadem dalšího tokenu z již dostupného kontextu. ⁽¹¹⁾

Vibe Coding

Způsob tvorby softwaru, při kterém člověk iteruje pomocí pokynů v přirozeném jazyce bez průběžné kontroly vygenerovaného kódu. ⁽²⁾

Vkládání kontextu (Context Injection)

V této práci označuje Context Injection cílené vložení relevantních informací do aktivního kontextu až v okamžiku, kdy jsou potřebné pro aktuální krok. ⁽⁷⁾

Vyvolávání nástrojů (Tool Calling)

Mechanismus, kterým model místo běžné textové odpovědi vybere konkrétní **Nástroje (Tools)** a vytvoří strukturované argumenty pro jeho vyvolání. ⁽⁹⁾

Vývoj řízený specifikací (Spec-Driven Development)

Prístup k AI-asistovanému vývoji, ve kterém explicitní specifikace řídí plánování, implementaci a ověřování změny. ⁽⁴⁶⁾

Větev (Branch)

Oddělená linie vývoje v systému správy verzí, která ukazuje na vlastní posloupnost commitů. ⁽⁴⁹⁾

Člověk ve smyčce (HITL)

Uspořádání automatizovaného procesu, ve kterém člověk v určených bodech poskytuje zpětnou vazbu, schválení nebo rozhodnutí. ⁽⁶¹⁾